studeersnel

BTL - HcsdlĐPT - 9 - hcsdldpt

Xử lý dữ liệu đa phương tiện (Học viện Công nghệ Bưu chính Viễn thông)



Scan to open on Studeersnel

Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)
Studocu is not sponsored or endorsed by any college or university

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

HỆ CSDL ĐA PHƯƠNG TIỆN

ĐỀ TÀI: XÂY DỰNG HỆ CSDL LƯU TRỮ VÀ NHẬN DẠNG ẢNH CHIM

Nhóm lớp: 2 - Nhóm BTL: 9

Giảng viên: Nguyễn Đình Hóa

Thành viên: Nguyễn Duy Hoàng - B22DCCN334

Trần Trọng Đại - B22DCCN178

Lê Đức Toàn - B22DCCN730

Hà Nội - 2026

studeersnel

Scan to open on Studeersnel

Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Mục lục

I. Xây dựng bộ dữ liệu ảnh chim ----------2

1. Nguồn dữ liệu ----------2

2. Hình ảnh ----------3

II. Thuộc tính và kỹ thuật trích rút đặc trưng ảnh ----------5

1. Nhận xét về bộ dữ liệu ----------5

2. Xây dựng bộ thuộc tính ảnh ----------6

2.1. Thuộc tính sử dụng về hình dạng ----------6

2.2. Thuộc tính sử dụng về màu sắc ----------13

2.3. Thuộc tính sử dụng về kết cấu ----------18

2.4. GLCM (Gray Level Co-occurrence Matrix) ----------20

2.5. Hu Moments ----------20

III. Lưu trữ, quản lý và tìm kiếm trong hệ CSDL ----------21

1. Lưu trữ và quản lý siêu dữ liệu ----------21

2. Kỹ thuật phân loại, tìm kiếm ảnh ----------22

IV. Xây dựng hệ thống nhận dạng và tìm kiếm ảnh ----------26

1. Sơ đồ khối của hệ thống ----------26

2. Quy trình thực hiện ----------26

3. Cài đặt các bước trung gian ----------27

3.1. Trích xuất đặc trưng hình ảnh ----------28

3.2. Phân cụm và đánh chỉ mục ngược ----------34

3.3. Tìm kiếm ảnh tương đồng ----------36

V. Kết quả kiểm nghiệm ----------40

1. Kết quả quá trình tìm kiếm ----------40

2. Đánh giá kết quả ----------41

studeersnel

Scan to open on Studeersnel

Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Yêu cầu

Xây dựng hệ CSDL lưu trữ và tìm kiếm ảnh chim:

1. Hãy xây dựng/sưu tầm một bộ dữ liệu ảnh gồm ít nhất 500 files ảnh chim khác nhau, các ảnh có cùng kích thước, vật trong ảnh có cùng tỉ lệ khung hình, tất cả đối tượng trong ảnh đều đang đậu, không bay, và cùng góc chụp ngang (SV tùy chọn định dạng ảnh).

2. Hãy xây dựng một bộ thuộc tính để nhận diện ảnh chim từ bộ dữ liệu đã thu thập (các đặc trưng thể hiện sự tương đồng và các đặc trưng thể hiện sự khác nhau giữa các ảnh chim). Trình bày chi tiết về lý do lựa chọn và giá trị thông tin của các thuộc tính được sử dụng.

3. Xây dựng hệ CSDL để quản lý các siêu dữ liệu đã có, trình bày cơ chế tìm kiếm ảnh chim tương đồng từ bộ siêu dữ liệu đó.

4. Xây dựng hệ thống tìm kiếm ảnh chim với đầu vào là một ảnh mới về một loài chim nào đó (ảnh của loài đã có và không có trong dữ liệu), đầu ra là 5 ảnh giống nhất, xếp thứ tự giảm dần về độ tương đồng nội dung với ảnh đầu vào.

a. Trình bày sơ đồ khối của hệ thống và quy trình thực hiện yêu cầu của đề bài.

b. Trình bày các kết quả trung gian của quá trình tìm kiếm ảnh chim tương đồng nói trên.

5. Demo hệ thống và đánh giá kết quả đã đạt được.

I. Xây dựng bộ dữ liệu ảnh chim

1. Nguồn dữ liệu

- Miêu tả bộ dữ liệu: Nhóm thu thập bộ dữ liệu gồm tổng cộng hơn 1000 ảnh chim, bao gồm nhiều loài chim khác nhau và được lựa chọn kỹ lưỡng trong tập dataset sau: Bird 525 Specise | Kaggle
- Kích thước của mỗi ảnh là 224x224 và đã được xóa nền.
- Định dạng tệp hình ảnh là "*.png".
- Các ảnh chim cùng loại được chụp ở cùng góc chụp ngang, đang đậu, không bay.

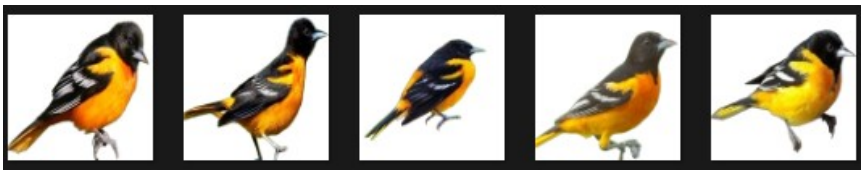
studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

2. Hình ảnh: Gồm nhiều loài chim khác nhau, bao gồm một số loài chim cụ thể:

- AFRICAN OYSTER CATCHER



- BALTIMORE ORIOLE



- CASPIAN TERN



- CUBAN TODY



- GREEN JAY



- GREEN WINGED DOVE



- KILLDEAR



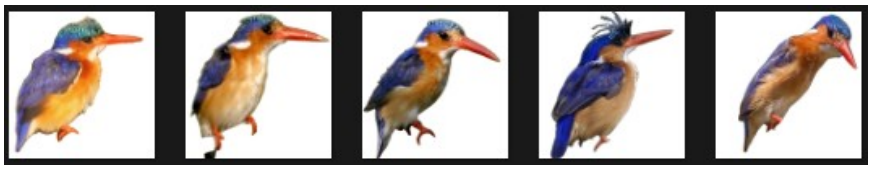
- LAZULI BUNTING

studeersnel

Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)



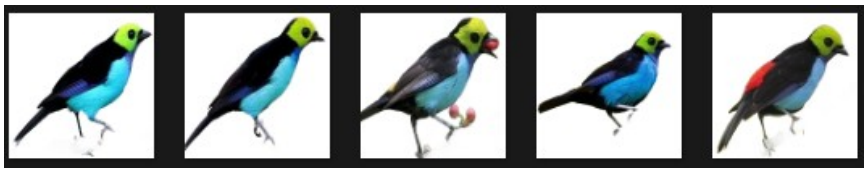
- MALACHITE KINGFISHER



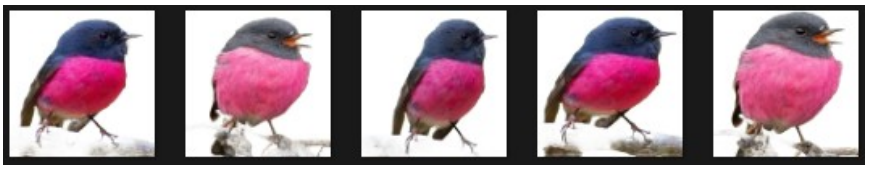
- PAINTED BUNTING



- PARADISE TANAGER



- PINK ROBIN



- RUFUOS MOTMOT



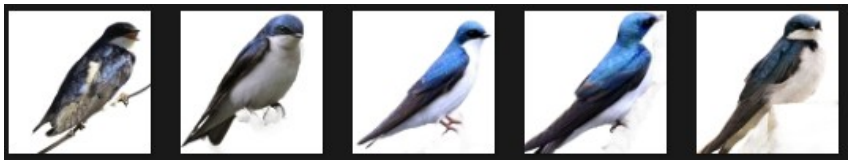
- SCARLET CROWNED FRUIT DOVE



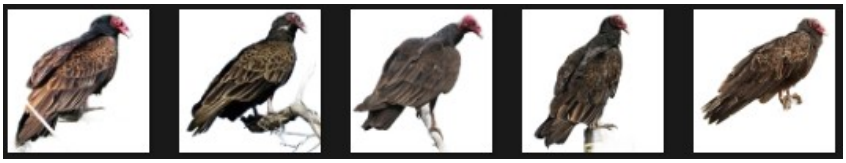
- TREE SWALLOW

studeersnel

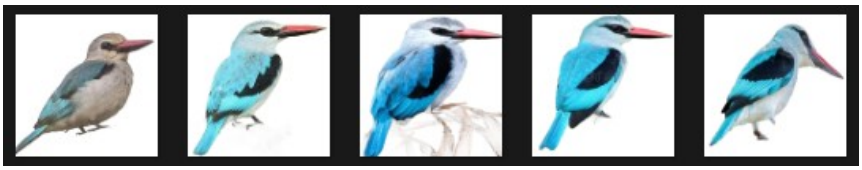
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)



- TURKEY VULTURE



- WOODLAND KINGFISHER



II. Thuộc tính và kỹ thuật trích rút đặc trưng ảnh

1. Nhận xét về bộ dữ liệu:

- Mỗi bức ảnh chỉ có một loài chim, đa số các bức ảnh có chim chiếm khoảng 50-75% diện tích ảnh và đều đã được xóa nền.

- Các ảnh chim cùng loài được chụp ở 1 vài góc độ, nhưng đa phần 85-90% là chụp hướng ngang, phần lớn các loài chim trong tập dữ liệu sở hữu các mảng màu trơn, lông mượt, ít vằn vện phức tạp. Mục đích là có thể nhìn thấy hình dáng và phân bố màu sắc mỗi loài một cách dễ dàng.

- Các hình ảnh của cùng 1 loài chim có màu sắc giống nhau tuy nhiên có thể về độ bão hòa ảnh và độ sáng khác nhau do cách chụp khác nhau. Các loài chim khác nhau thì có thể có màu sắc trùng với nhau (ví dụ như AFRICAN OYSTER CATCHER

và CASPIAN TERN, SRI LANKA BLUE MAGPIE đều có mào màu đỏ)

- Các hình ảnh của cùng 1 loài chim cũng sẽ có hình dáng giống nhau. Tuy nhiên hình dáng của các loài chim khác nhau lại không khác hoàn toàn (Ví dụ như PARADISE TANAGER và TREE SWALLOW có hình dáng kha khá giống nhau).

- Sự kết hợp giữa yếu tố màu sắc và hình dáng sẽ tạo nên đặc trưng của các loài chim và có thể phân biệt các loài chim với nhau.

=> Hướng trích rút tập trung áp dụng đặc trưng về màu sắc và hình dạng.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

2. Xây dựng bộ thuộc tính ảnh

Mắt người thường phân biệt mỗi loài dựa vào màu sắc và hình dạng => Do đó ta sử dụng hướng trích rút đặc trưng là màu sắc (phương pháp Color Histogram) và trích rút đặc trưng hình dạng (phương pháp HOG).

2.1. Thuộc tính sử dụng về hình dạng

Thuật toán HOG (Histogram of Oriented Gradient)

Hình dạng có khuynh hướng chỉ đến một khu vực đặc biệt trong ảnh. Thuật toán HOG được sử dụng chủ yếu để mô tả hình dạng và sự xuất hiện của một đối tượng trong ảnh. Sử dụng thông tin về sự phân bố của các cường độ Gradient (intensity gradient) hoặc của hướng biên (edge directions) để mô tả các đối tượng cục bộ trong ảnh. HOG chia hình ảnh đầu vào thành các ô vuông nhỏ (cell) và sau đó tính toán histogram và hướng của gradient cho các điểm nằm trong cell. Để tăng cường hiệu năng nhận dạng, các histogram cục bộ có thể được chuẩn hóa.

Có 5 bước cơ bản để xây dựng một vector HOG cho hình ảnh, bao gồm:

- Tiền xử lý
- Tính gradient
- Tính vector đặc trưng cho từng ô (cells)
- Chuẩn hóa khối (blocks)
- Tính toán vector HOG

Bước 1: Tiền xử lý

Thuật toán HOG tập trung vào cấu trúc hình học (đường nét, góc cạnh) thay vì màu sắc. Do đó, ảnh màu RGB kích thước 224 x 224 sẽ được chuyển thành ảnh xám (Grayscale).

Phép toán: Với mỗi pixel có giá trị màu (R, G, B), cường độ sáng xám Y được tính bằng công thức: $Y = 0.299*R + 0.587*G + 0.114*B$

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

1. Ảnh gốc (224x224)



2. Ảnh xám (Bước 1)



Bước 2: Tính gradient

Phương pháp sẽ lọc ma trận cường độ ảnh với các bộ lọc Sobel mask.

Để tính bộ lọc Sobel, thực hiện tính tích chập của mặt nạ với hình ảnh ban đầu. Ký hiệu ảnh I là ma trận ảnh gốc và Gx, Gy là hai ma trận ảnh mà mỗi điểm trên nó lần lượt là đạo hàm theo trục x và trục y của ảnh. Kí hiệu * là phép nhân tích chập giữa bộ lọc bên trái và đầu vào bên phải.

Đây là bước đầu tiên, được thực hiện bằng hai phép nhân chập ảnh gốc với 2 chiều (quét qua từng pixel trong ảnh xám), tương ứng với các toán tử lấy đạo hàm theo hai hướng Ox và Oy. Trong đó, 2 hướng tương ứng đó là:

- Kernel ngang (tính đạo hàm theo trục x): Dx = [-1, 0, 1]

- Kernel dọc (tính đạo hàm theo trục y): Dy = [-1, 0, 1]^T

Nếu có một ảnh input là I, ta sẽ có 2 ảnh đạo hàm riêng theo 2 hướng đó, theo công thức: Gx = I * Dx và Gy = I * Dy.

Giá trị độ lớn (Magnitude) gradient và phương (Direction) gradient có thể được tạo ra từ 2 đạo hàm Gx và Gy theo công thức bên dưới:

- Độ lớn: $g = \sqrt{g_x^2 + g_y^2}$

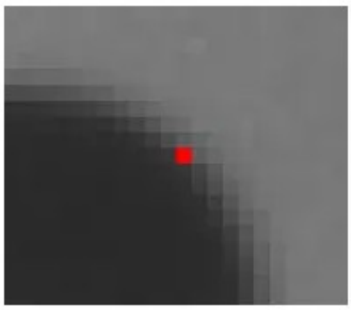
- Phương gradient: Xác định phương thay đổi cường độ màu sắc hay chính là phương đổ bóng của hình ảnh: $\theta = \arctan \frac{g_y}{g_x}$

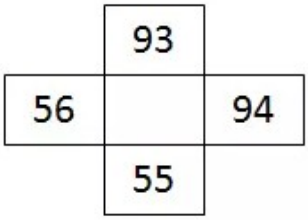
studeersnel

Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Vậy mỗi bức ảnh giờ đây đã được lấy các đặc trưng biểu diễn thông qua 2 thông số liên quan đến mức độ thay đổi cường độ màu sắc (ma trận độ lớn Gradient) và hướng thay đổi cường độ màu sắc (ma trận phương Gradient).

- Ví dụ : Chúng ta sẽ áp dụng các công thức trên để tính được gradient của điểm ảnh này:





$$G_x = I * D_x = [56\ x\ 94\ ] * [-1\ 0\ 1\ ] = [38]$$

$$G_y = I * D_y = [93\ y\ 55\ ] * [-1\ 0\ 1\ ] = [-38]$$

$$G = \sqrt{G_x{}^2 + G_y{}^2} = \sqrt{38^2 + (-38)^2} \approx 53.74$$

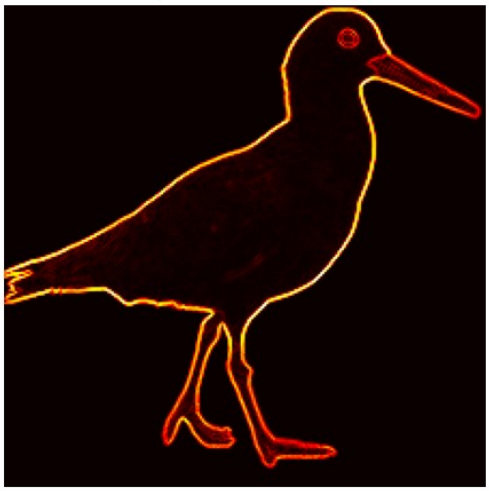
$$\theta = argtan\frac{G_y}{G_x} = argtan\frac{-38}{38} = 45\ \text{deg.}$$

Kết quả thu được sau khi thực hiện bước 2:

2. Ảnh xám (Bước 1)



3. Độ lớn Gradient (Bước 2)



Bước 3: Tính vector đặc trưng cho từng ô (cells)

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Qua quá trình thực nghiệm cho thấy, nếu ta sử dụng cell 8 x 8, thuật toán HOG sẽ soi xét bức ảnh ở mức độ rất chi tiết (vị trí mỏ, nếp gấp cánh). Điều này khiến mô hình cực kỳ nhạy cảm và dễ nhận dạng sai khi con chim thay đổi tư thế (VD: đang đứng thẳng so với cúi ngậm thức ăn). Bằng cách tăng kích thước cell lên 16x16, thuật toán sẽ bỏ qua tiểu tiết để tập trung vào hình dáng tổng thể của loài chim.

Ảnh được chia thành các ô nhỏ (cell) để phân tích cục bộ. Với ảnh kích thước 224 x 224, số lượng cell thu được là (224 / 16) * (224 / 16) = 14 * 14 cells. (Tổng cộng 196 cells). Mỗi cell chứa 256 pixels (16 x 16).

Đặc trưng của mỗi bức ảnh được biểu diễn thông qua 2 thông số, đó là mức độ thay đổi cường độ màu sắc (ma trận gradient magnitude) và hướng thay đổi cường độ màu sắc (ma trận gradient direction). Do đó cần tạo ra được một bộ mô tả (feature descriptor) sao cho biến đổi bức ảnh thành một véc tơ mà thể hiện được cả 2 thông tin này. Một biểu đồ Histogram thống kê độ lớn Gradient sẽ được tính trên mỗi ô cục bộ. Véc tơ histogram sẽ được tạo ra như sau:

+ Bước 1: Chia các bins tương ứng của phương gradient:

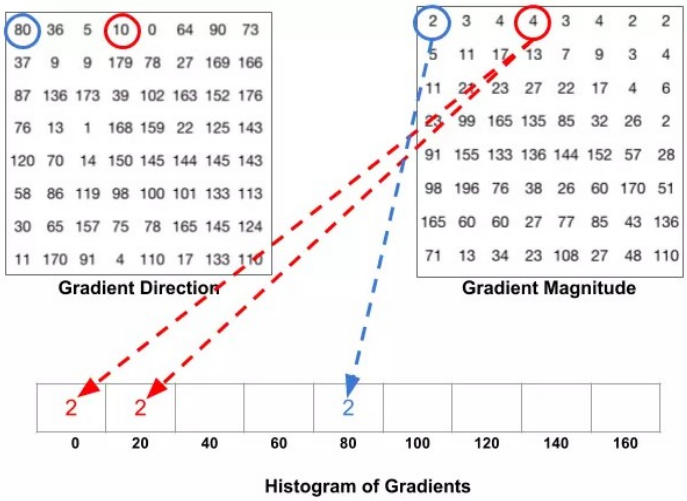
- Chẳng hạn chia thành 9 bins. Độ lớn của phương gradient sẽ nằm trong khoảng [0, 180] nên mỗi bins sẽ có độ dài là 20.

+ Bước 2: Ánh xạ độ lớn gradient vào các bins tương ứng của phương gradient.

- Mỗi một phương gradient sẽ ghép cặp với một độ lớn gradient ở cùng vị trí tọa độ. Khi biết được phương gradient thuộc bins nào trong vector bins, ta sẽ điền vào giá trị của độ lớn gradient vào chính bin đó.
- Đầu mút là các giá trị chia hết cho độ rộng của một bin (chẳng hạn 0, 20, 40,… là những đầu mút bin).

Ví dụ: Ở hình bên dưới, ô viền xanh có hướng 80 độ và cường độ là 2, ta điền 2 vào bin thứ 5. Với ô bao quanh màu đỏ. có hướng 10 độ và cường độ 4. Vì không có bin 10 độ, nên ta cho bin 0 độ và 20 độ, mỗi bin thêm 2 đơn vị.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

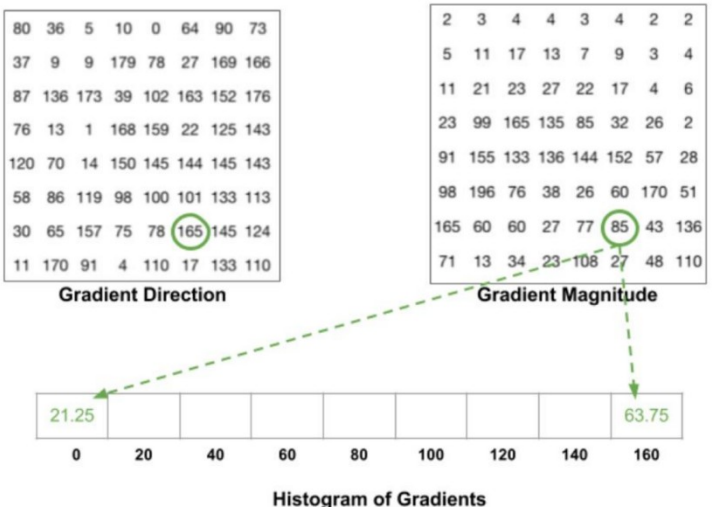


- Trong trường hợp độ lớn phương gradients không rơi vào các đầu mút, ta sẽ sử dụng linear interpolation (nội suy tuyến tính) để phân chia độ lớn gradient về 2 bins liền kề mà giá trị phương gradient rơi vào.

Cụ thể: giá trị phương gradient bằng x ghép cặp với độ lớn gradient bằng y, x ∈ [x0, x1] tức là phương gradients rơi vào khoảng giữa bin thứ (i - 1) và bin thứ i. Khi đó tại 2 bins (i - 1) và i được điền vào giá trị cường độ theo công thức sau:

Giá trị tại bins L - 1: $x_{l-1} = \frac{(x_1 - x)}{(x_1 - x_0)} * y$

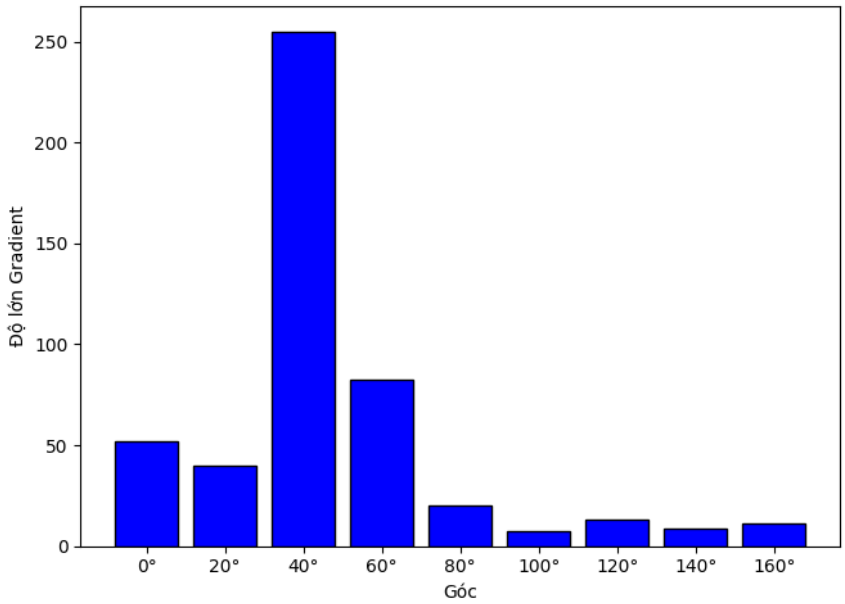
Giá trị tại bins L: $x_l = \frac{(x - x_0)}{(x_1 - x_0)} * y$



+ Bước 3: Tính tổng tất cả các độ lớn gradient thuộc cùng 1 bins của véc tơ bins ta thu được biểu đồ Histogram of Gradients.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Kết quả thu được sau khi thực hiện bước 3:



Bước 4: Chuẩn hóa khối (blocks)

Lý do chuẩn hóa là bởi vector histogram sẽ phụ thuộc vào cường độ các pixels nên có thể ảnh có cùng nội dung nhưng ảnh sáng ảnh tối khác nhau, giá trị histogram cũng khác nhau. Vì vậy ta cần chuẩn hóa vector histogram để cả hai bức ảnh có cùng một vector biểu diễn.

+ Quá trình chuẩn hóa sẽ thực hiện trên một block kích thước 2x2 trên lưới ô vuông ban đầu. Ở đây, chúng ta sẽ kết hợp bốn ô 8×8 để tạo một khối 16×16. Như vậy chúng ta sẽ có 4 vector histogram (mỗi block chứa 4 cells) kích thước 1x9 (mỗi cell có 9 giá trị bins), tổng hợp các vector sẽ thu được vector histogram tổng hợp kích thước là 1x36 và sau đó chuẩn hóa theo norm chuẩn bậc 2 trên vector này.

Để chuẩn hóa ma trận này, chúng ta sẽ chia từng giá trị này cho căn bậc hai của tổng bình phương các giá trị. Về mặt toán học, đối với một vector V đã cho:

$$V^{-} = \{a_1, a_2, a_3, \dots, a_{36}\}$$

Chúng ta tính toán căn bậc 2 của tổng bình phương:

$$|V^{-}| = \sqrt{{a_1}^2 + {a_2}^2 + {a_3}^2 + \cdots + {a_{36}}^2}$$

Và chia tất cả các giá trị trong vector V với giá trị này.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

=> Vector chuẩn hóa: $V^{-} = \{\frac{a_1}{|V^{-}|}, \frac{a_2}{|V^{-}|}, \frac{a_3}{|V^{-}|}, \dots, \frac{a_{36}}{|V^{-}|}\}$

Khi đó vector v đã được chuẩn hóa sao cho mỗi chiều có giá trị nằm trong [0;1]. Kết quả sẽ là một vectơ chuẩn hóa có kích thước 1x36.

+ Với v là vector cần chuẩn hóa chứa tất cả các histogram của một khối. ||v(k)|| là giá trị chuẩn hóa của v theo các chuẩn k = 1, 3 và e là một hằng số nhỏ. Ta có thể chuẩn hóa vector histogram của các gradient theo norm 1, hoặc norm 2.

Ở đây chúng ta dùng norm 2: $f = \frac{v}{\sqrt{\|v_2\|^2 + e^2}}$

Bước 5: Tính toán vector HOG

Ghép các vector đặc trưng khối sẽ thu được vector đặc trưng HOG cho ảnh. Số chiều vector đặc trưng ảnh tính theo công thức:

$$size_{image} = n * size_{block}$$

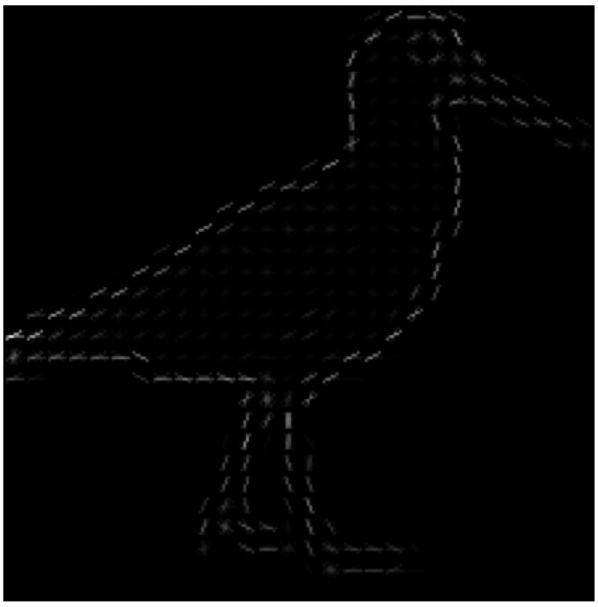
trong đó :

n là số khối của hình ảnh.

$size_{block}$ là số chiều của vector đặc trưng khối

Đối với ảnh trong dữ liệu, số block: (14 - 2 + 1) * (14 - 2 + 1) = 13 * 13 = 169 Blocks. Mỗi block có 36 chiều => Chiều dài Vector HOG của bức ảnh: 169 * 36 = 6084 chiều (dimensions), kích thước: 6084 x 1.

5. Biểu diễn Đặc trưng HOG (Bước 5)



studeersnel

Scan to open on Studeersnel

Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

2.2. Thuộc tính sử dụng về màu sắc

Kỹ thuật trích rút Color Histogram

Color histogram thống kê số lượng các pixel có giá trị nằm trong một khoảng màu nhất định cho trước (RGB hoặc HSV) với mỗi kênh màu cố định, chia kênh màu này thành n bin. Sau đó, thống kê số lượng pixel trên ảnh có giá trị màu thuộc về "n bin" này. Cuối cùng, nối các bin của các kênh màu lại với nhau để tạo thành đặc trưng.

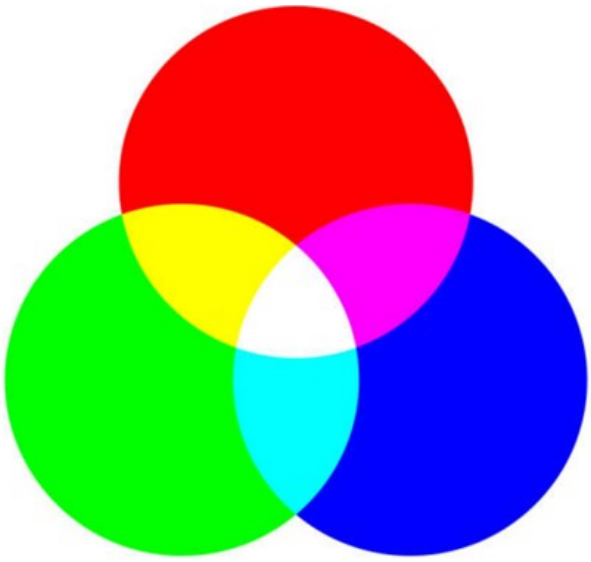
Ưu điểm:

- Tính toán nhanh, đơn giản, thích hợp trong các ứng dụng thời gian thực.
- Khi ảnh bị xoay đi thị phân phối màu hầy như là không đổi.

Nhược điểm:

- Không nói lên được sự tương đồng về hình dáng, cấu trúc ảnh.
- Dễ bị nhiễu với thanh đổi về cường độ sáng.

+ Hệ màu RGB:



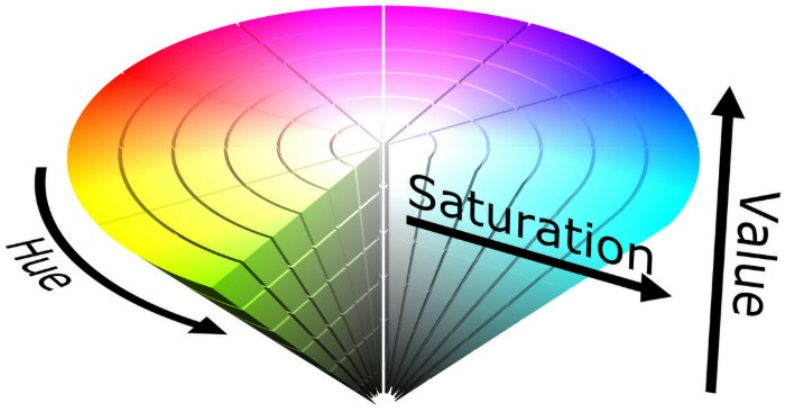
RGB là không gian màu phổ biến dùng trong máy tính, máy ảnh, điện thoại và nhiều thiết bị kỹ thuật số khác nhau. Không gian màu này khá gần với cách mắt người tổng hợp màu sắc. Nguyên lý cơ bản là sử dụng 3 màu sắc cơ bản R (red - đỏ), G (green - xanh lục) và B (blue - xanh lam) để biểu diễn tất cả các màu sắc.

Thông thường, trong mô hình 24 bit mỗi kênh màu sẽ sử dụng 8 bit để biểu diễn, tức là giá trị R, G, B nằm trong khoảng 0 - 255. Bộ 3 số này biểu diễn cho từng

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

điểm ảnh, mỗi số biểu diễn cho cường độ của một màu. Với mô hình màu 24 bit thì số màu tối đa có thể tạo ra là 255 x 255 x 255 = 16581375 màu.

+ Hệ màu HSV:



Không gian màu HSV là một cách tự nhiên hơn để mô tả màu sắc:

- Hue (Tông màu): Cho biết đây là màu gì? Là tổ hợp của 12 màu đậm nhạt khác nhau trên vòng tuần hoàn màu sắc, được biểu diễn dưới dạng một số từ 0 đến 360 độ.

- Saturation (Độ bão hòa): mô tả lượng màu xám trong một màu cụ thể, từ 0 đến 100 phần trăm. Giảm thành phần này về 0 sẽ tạo ra nhiều màu xám hơn và tạo hiệu ứng mờ dần.

- Value (Độ sáng): Giá trị hoạt động cùng với độ bão hòa và mô tả độ sáng hoặc cường độ của màu, từ 0 đến 100 phần trăm, trong đó 0 là màu đen hoàn toàn và 100 là màu sáng nhất và hiển thị nhiều màu nhất.

Bước 1: Chuyển đổi không gian màu (từ RGB => HSV)

Thông thường ảnh được thể hiện dưới dạng RGB, mỗi điểm ảnh là sự tổ hợp từ 3 màu Red, Green, Blue tạo thành. Các giá trị như nhau của RGB có thể mô tả màu tương đối khác nhau trên các thiết bị khác nhau điều này sẽ làm ảnh hưởng đến kết quả bài toán nhận diện ảnh dựa trên màu sắc.

Do ảnh chim có duy nhất một loài chim và các ảnh sẽ khác nhau về góc chụp, độ sáng, màu sắc, môi trường. Vì vậy, thay vì sử dụng mô hình màu RGB, ta sử dụng mô hình màu HSV để tăng độ chính xác cho bài toán. Cho nên ta cần chuyển đổi ảnh RGB sang HSV.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Ta thực hiện chuyển đổi tuyến tính từ RGB sang HSV theo công thức dưới đây:

$$r' = \frac{r}{255}, g' = \frac{g}{255}, b' = \frac{b}{255}$$

$$C_{max} = \max(r', g', b') \qquad C_{min} = \min(r', g', b')$$

$$\Delta = C_{max} - C_{min}$$

$$H = \begin{cases} \frac{g' - b'}{\Delta} \ when\ C_{max} = v' \\ 2 + \frac{(b' - r')}{\Delta} \ when\ C_{max} = g' \\ 4 + \frac{r' - g'}{\Delta} \ when\ C_{max} = b' \end{cases}$$

$$H = \left(\frac{H}{6}\right) \%1.0$$

$$s = \begin{cases} 0 \ when\ \Delta = 0 \\ \frac{\Delta}{v} \ when\ \Delta ! = 0 \end{cases}$$

$$v = C_{max}$$

$$\Rightarrow \ H = H * 180$$
$$\Rightarrow \ s = s * 255$$
$$\Rightarrow \ v = v * 255$$

Áp dụng với từng pixel, ta sẽ được kết quả ảnh HSV từ ảnh RGB như bên dưới:

1. Ảnh gốc RGB



2. Không gian màu HSV



Bước 2: Phân mảnh và tạo mặt nạ khử nền (Background masking)

Do các ảnh trong tập dữ liệu đều có phông nền trắng, nếu đưa toàn bộ ảnh vào tính toán, màu trắng của nền sẽ áp đảo màu sắc của chim. Vì vậy, ta tiến hành tạo mặt nạ nhị phân (Binary mask) để tách vật thể:

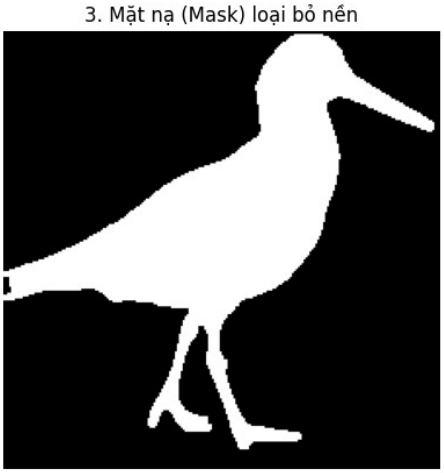
Ảnh được chuyển sang mức xám (Grayscale).

Áp dụng ngưỡng (Thresholding) để phân loại điểm ảnh:

$$Mask(x, y) = \begin{cases} 1 & \text{nếu } I_{gray}(x, y) < 240 \text{ (Thuộc về chim)} \\ 0 & \text{nếu } I_{gray}(x, y) \geq 240 \text{ (Thuộc về nền trắng)} \end{cases}$$

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Ở các phép toán tiếp theo, hệ thống chỉ sử dụng các điểm ảnh có giá trị Mask = 1.



Bước 3: Lượng tử hóa không gian màu (Binning)

Thay vì đếm từng giá trị màu riêng lẻ (vốn có tới hàng triệu màu khác nhau), ta chia không gian HSV thành các bins rời rạc, mỗi bin đại diện cho một dải màu cụ thể. Sử dụng bộ bins giảm thiểu được độ phức tạp tính toán, ngoài ra giảm thiểu độ nhiễu, giảm số chiều tính toán nhưng vẫn đem lại hiệu quả tốt.

Ta chia bins theo tỉ lệ (H,S,V) - (16,4,3) => Số chiều: 16 * 4 * 3 = 192. Trong tập dữ liệu ảnh chim, màu sắc lông chim là yếu tố phân loại quan trọng, nhưng thường bị ảnh hưởng bởi độ sáng và bóng râm của môi trường. Bằng cách thiết lập cấu hình bins là (16, 4, 3), ta tập trung độ phân giải tối đa cho H để bắt được các dải màu tinh tế (như phân biệt màu đỏ và cam). Đồng thời, nén S và V để giúp hệ thống miễn nhiễm với các nhiễu loạn về ánh sáng, cho phép nhận diện đúng màu lông chim bất kể ảnh chụp ngoài nắng hay trong bóng râm.

Sau khi trích rút, ta thu được vector đặc trưng có số phần tử là 16 x 4 x 3 = 192.

Bước 4: Tính toán lược đồ màu (Histogram Calculation)

Hệ thống quét qua toàn bộ các điểm ảnh của con chim (những nơi Mask = 1) và đếm số lượng điểm ảnh rơi vào từng bin.

Lược đồ h tại một bin có tọa độ (i, j, k) được tính bằng tổng số pixel thỏa mãn điều kiện màu sắc:

$$h(i,j,k) = \sum_{x=0}^{223}\sum_{y=0}^{223} \begin{cases} 1 & \text{nếu pixel}(x,y) \in \text{bin}(i,j,k) \text{ và Mask}(x,y) = 1 \\ 0 & \text{trường hợp khác} \end{cases}$$

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

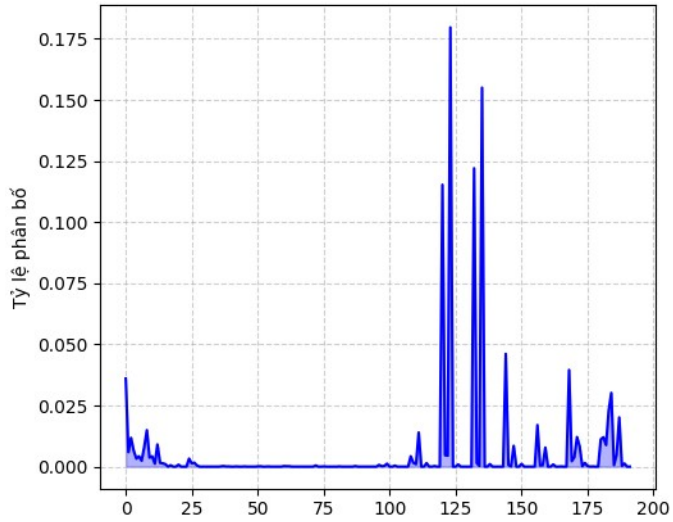


Bước 5: Chuẩn hóa vector (L1 Normalization)

Do kích thước con chim trong các bức ảnh có thể to nhỏ khác nhau, để so sánh công bằng, ta phải chuyển số đếm (pixel count) thành tỷ lệ phần trăm (phân bố xác suất). Gọi vector lược đồ màu là v, tổng số điểm ảnh là N. Vector chuẩn hóa v(norm) được tính bằng: $v_{norm} = \frac{v}{N}$

(Đảm bảo tổng tất cả các phần tử trong vector bằng 1).

Kết quả cuối cùng: Đầu ra là một mảng 1 chiều (Vector) gồm 192 số thực có giá trị từ 0 đến 1.



2.3. Thuộc tính sử dụng về kết cấu

Local Binary Patterns (LBP)

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

LBP là một toán tử đơn giản nhưng rất hiệu quả, nó gắn nhãn các pixel của hình ảnh bằng cách phân ngưỡng các pixel xung quanh và đưa ra ảnh kết quả dưới dạng nhị phân. Do khả năng phân tách tốt và phi phí tính toán thấp, toán tử LBP đã trở thành một phương pháp phổ biến trong các ứng dụng khác nhau.

Tính chất quan trọng nhất của LBP trong các ứng dụng thực tế là sự chống chịu của nó đối với các thay đổi mức xám (VD: Biến đổi về ánh sáng). Một tính chất quan trọng khác là tính toán đơn giản, phục vụ cho việc xử lý hình ảnh trong thời gian thực.

+ Quy tắc tìm LBP của một hình ảnh như sau:

- Đặt giá trị pixel làm pixel trung tâm.

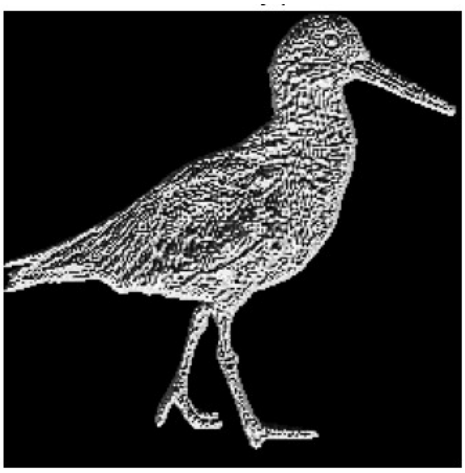
- Thu thập các pixel lân cận của nó (thông thường lấy ma trận 3 x 3; tổng số pixel lân cận là 8 => radius (bán kính) = 1; n_points = 8)

- Đặt ngưỡng giá trị pixel lân cận của nó thành 1 nếu giá trị của nó lớn hơn hoặc bằng giá trị pixel trung tâm. Nếu không, đặt ngưỡng đó thành 0.

- Sau khi tạo ngưỡng, thu thập tất cả các giá trị ngưỡng từ vùng lân cận theo chiều kim đồng hồ hoặc ngược chiều kim đồng hồ. Bộ sưu tập sẽ cung cấp mã nhị phân gồm 8 chữ số. Chuyển đổi mã nhị phân thành số thập phân.

- Thay thế giá trị pixel trung tâm bằng số thập phân kết quả và thực hiện quy trình tương tự cho tất cả các giá trị pixel có trong hình ảnh.

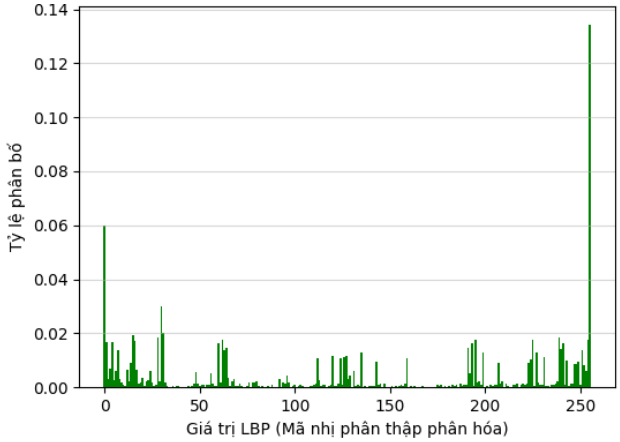
(Khi áp dụng thuật toán LBP cho tập dữ liệu ảnh trên, ta cần tạo Mask để khử nền giống như Color Histogram, do các ảnh đều có phông nền trắng).



- Cuối cùng, ta chuyển số đếm (pixel count) thành tỷ lệ phần trăm (phân bố xác suất) bằng chuẩn hóa vector (L1).

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Kết quả cuối cùng: Đầu ra là mảng 1 chiều (vector) gồm 256 số thực có giá trị từ 0 đến 1.



+ Ưu điểm:

- Đơn giản và chi phí tính toán thấp: So với các đặc trưng khác như SIFT hay SURF, LBP yêu cầu ít bước tính toán hơn và các các bước làm cũng đơn giản hơn.

- Chống chịu với thay đổi độ sáng: Do LBP được cấu trúc từ mức xám tương đối giữa các pixel (hiệu của pixel láng giềng và pixel trung tâm) nên không bị ảnh hưởng khi mức xám biến đổi do chiếu sáng.

+ Nhược điểm:

- Không chống chịu với ảnh xoay: Khi hình ảnh xoay, các giá trị mức xám sẽ di chuyển dọc theo chu vi của vòng tròn xung quanh pixel trung tâm. Điều này khiến chuỗi nhị phân cũng xoay theo và cho ra giá trị khác.

- Độ dài vector đặc trưng tăng theo hàm số mũ khi số lượng pixel láng giềng tăng: Khi xét n pixel láng giềng, độ dài của vector đặc trưng (histogram) là 2n sẽ tăng nhanh khi n tăng.

- Làm mất thông tin về cấu trúc không gian: Toán tử LBP có một nhược điểm trong nhận dạng là xét vùng không gian nhỏ, do đó phép so sánh bit được thực hiện giữa hai giá trị pixel đơn lẻ bị ảnh hưởng nhiều bởi nhiễu. Hơn nữa, các đặc trưng được tính toán trong vùng lân cận 3 × 3 cục bộ không thể nắm bắt cấu trúc tỷ lệ lớn hơn (cấu trúc vĩ mô) có thể là đặc điểm nổi trội của loài chim.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

2.4. GLCM (Gray Level Co-occurrence Matrix)

GLCM là một phương pháp thống kê để kiểm tra kết cấu xem xét mối quan hệ không gian của các pixel. Các hàm GLCM mô tả kết cấu của hình ảnh bằng cách tính toán tần suất xuất hiện của các cặp pixel với các giá trị cụ thể và trong một mối quan hệ không gian cụ thể trong một hình ảnh, tạo GLCM, sau đó trích xuất các biện pháp thống kê từ ma trận này.

GLCM sẽ thường trích xuất 4 thuộc tính: Contrast, Dissimilarity, Homogeneity, Energy. Các loài chim màu đen tuyền, bóng mượt sẽ có chỉ số Homogeneity cực cao. Ngược lại, các loài chim có lông ngực xù xì, tơi xốp sẽ có chỉ số Contrast và Entropy cao. GLCM bổ sung cho những thông tin kết cấu mà LBP bị lọt lưới.

+ Cách thức tính:

- Chọn vùng quan tâm trong ảnh để tính toán ma trận GLCM.
- Xác định các thông số như kích thước cửa sổ, bước nhảy và số cấp độ để tính toán ma trận GLCM.
- Tính toán ma trận GLCM cho mỗi hướng bằng cách đếm số lần xuất hiện của các cặp giá trị cường độ xung quanh nhau.
- Chuẩn hóa ma trận GLCM để đảm bảo tổng các giá trị trong ma trận bằng 1.
- Tính toán các đặc trưng hình ảnh từ ma trận GLCM, bao gồm độ đồng nhất, độ đa dạng, độ lệch chuẩn, độ phân tán và độ liên kết.
- Lựa chọn các đặc trưng hình ảnh để sử dụng trong các ứng dụng phân loại và nhận dạng hình ảnh.

+ Ưu điểm: Cung cấp thông tin về mức độ tương quan giữa các pixel trong ảnh

Đơn giản, dễ sử dụng, không yêu cầu định dạng ảnh cụ thể

+ Nhược điểm: Nhạy cảm với nhiễu, phụ thuộc vào kích thước cửa sổ

Chỉ áp dụng được cho ảnh xám, khó xử lý các vùng không đồng nhất.

2.5. Hu Moments

Hu Moments (Mômen Hu) là tập hợp 7 mô-men ảnh bất biến được sử dụng rộng rãi trong thị giác máy tính để trích xuất đặc trưng hình dạng của đối tượng. Đặc

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

điểm nổi bật của Hu Moments là tính bất biến đối với phép tịnh tiến, quay và thay đổi tỷ lệ, giúp nhận dạng hình ảnh chính xác ngay cả khi đối tượng bị biến đổi hình học.

Trong đó, mômen ảnh (m{pq}) dùng để mô tả phân bố cường độ điểm ảnh, xác định diện tích, trọng tâm và hình dạng. 7 mômen này (I1 đến I7) được kết hợp từ các mô-men chuẩn hóa cấp cao (u{pq}), tạo ra các giá trị đặc trưng duy nhất cho một hình dạng: I1: Độ bền vững về diện tích.

I2 => I7: Mô tả các đặc trưng phức tạp hơn như đối xứng, chiều dài, độ cong…

Quy trình trích xuất đặc trưng Hu Moments được thực hiện như sau:

- Tiền xử lý: Chuyển ảnh sang ảnh nhị phân (trắng / đen) để tách đối tượng và nền.
- Tính toán trọng tâm: Xác định ({x},{y})) để tính mômen trung tâm.
- Tính 7 mômen Hu: Áp dụng công thức tính (I1 đến I7) dựa trên các mômen đã chuẩn hóa. Hu Moments hoạt động hiệu quả nhất khi đối tượng có đường biên rõ ràng và ít bị nhiễu.

III. Lưu trữ và quản lý hệ CSDL

1. Lưu trữ và quản lý siêu dữ liệu

Để quản lý tập dữ liệu gồm nhiều ảnh chim, việc chỉ lưu trữ các file ảnh *.png trong thư mục là không đủ để phục vụ truy vấn. Hệ thống cần một CSDL có cấu trúc để lưu trữ các siêu dữ liệu (metadata) và các vector đặc trưng đã được trích xuất.

Ta sẽ sử dụng mô hình lưu trữ kết hợp:

+ File system (Hệ thống file vật lý):

- Lưu trữ các file ảnh gốc *.png trong các thư mục được phân loại theo tên loài.
- features.pkl: Vector đặc trưng của mỗi ảnh được lưu trữ bên trong, định dạng .pkl (pickle), tổ chức dưới dạng cấu trúc dữ liệu từ điển:

Key: Kiểu string, lưu trữ Image_ID của bức ảnh. (VD: 'AFRICAN OYSTER CATCHER (3)').

Value: Kiểu mảng numpy 1 chiều. Lưu trữ vector đặc trưng của bức ảnh đó.

Minh họa:{

'BIRD_(1)': array([0.012, 0.054, ... , 0.001]),

'BIRD_(2)': array([0.033, 0.011, ... , 0.089])

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

}

+ Relational Database (CSDL quan hệ): Một bảng dữ liệu mang tên Bird_Metadata được thiết kế với các trường thông tin sau:

Image_ID (Primary Key): Mã định danh duy nhất cho mỗi bức ảnh.

Species_Label (String): Nhãn dán phân loại (VD: "BALTIMORE ORIOLE", "CUBAN TODY").

File_Path (String): Đường dẫn trỏ tới file ảnh vật lý trên ổ cứng.

2. Kỹ thuật phân loại, tìm kiếm ảnh

2.1. Phân cụm K-means + Inverted File Index (IVF)

a. K-means

Phân cụm K-Means là một thuật toán học máy không giám sát có nhiệm vụ phân chia một tập dữ liệu gồm N phần tử thành K cụm rời rạc. Thuật toán hoạt động dựa trên nguyên tắc tối thiểu hóa tổng khoảng cách từ các điểm dữ liệu đến tâm của cụm (centroid) chứa nó. Trong bài toán xử lý ảnh, K-Means sẽ gom các bức ảnh có các vector đặc trưng tương đồng lại với nhau.

Trong các hệ thống truy xuất ảnh dựa trên nội dung, nếu áp dụng phương pháp quét tuyến tính, hệ thống phải so sánh ảnh truy vấn với toàn bộ N bức ảnh trong cơ sở dữ liệu. Độ phức tạp tính toán sẽ là O(N), khiến hệ thống bị quá tải và phản hồi chậm khi CSDL lên đến hàng chục ngàn bức ảnh.

Bằng cách chia N dữ liệu thành K cụm, số phép tính khoảng cách giảm từ O(N) xuống chỉ còn O(K + N / K). Hệ thống hoạt động tốt theo thời gian thực ngay cả với CSDL lớn.

+ Ưu điểm:

- Tốc độ truy vấn nhanh: Giảm độ phức tạp từ O(N) xuống còn O(K + N / K).

- Phù hợp với vector nhiều chiều: Hoạt động ổn định trên các vector đặc trưng có nhiều chiều.

- Dễ dàng mở rộng và cập nhật: Khi có thêm ảnh mới vào CSDL, chỉ cần tính khoảng cách đến các tâm cụm để xếp ảnh vào cụm tương ứng mà không cần phải huấn luyện lại toàn bộ mô hình từ đầu.

+ Nhược điểm:

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

- Đánh đổi độ chính xác: Có một xác suất rất nhỏ ảnh tương đồng nhất lại nằm ở rìa của một cụm lân cận khác, dẫn đến việc bỏ sót.

- Chi phí huấn luyện ban đầu: Thuật toán K-Means tốn khá nhiều thời gian để hội tụ khi khởi tạo ban đầu, đặc biệt khi số lượng cụm K lớn.

b. Inverted File Index (IVF)

IVF là một cấu trúc dữ liệu dạng từ điển, dùng để ánh xạ từ một cụm ra danh sách các phần tử thuộc cụm đó (cluster_id: [image_id]). Kỹ thuật này tương tự như mục lục ở cuối các cuốn sách, giúp hệ thống tra cứu ngay lập tức các phần tử trong một cụm mà không cần duyệt qua toàn bộ cơ sở dữ liệu.

Minh họa: {

0: ['BIRD_001', 'BIRD_015', 'BIRD_088'],

1: ['BIRD_002', 'BIRD_003']

}

+ Quá trình áp dụng K-Means + IVF trên hệ thống được chia làm 2 giai đoạn:

Giai đoạn 1: Lập chỉ mục:

- Thu thập vector: Lấy toàn bộ vector đặc trưng từ cơ sở dữ liệu.
- Khởi tạo K-Means: Thiết lập số lượng cụm K.
- Phân cụm: Thuật toán tự động lặp lại việc tìm các tâm điểm và gán các vector vào tâm gần nhất cho đến khi mạng hội tụ. Cập nhật vị trí của K tâm cụm.
- Xây dựng IVF: Duyệt qua nhãn của từng bức ảnh sau khi gom cụm. Ánh xạ định danh của bức ảnh (Image_ID) vào một danh sách thuộc quản lý của cụm tương ứng để tạo thành chỉ mục ngược.
- Lưu trữ: Xuất file ma trận tâm cụm (kmeans_model.pkl) và chỉ mục ngược (ivf_index.pkl) vào bộ nhớ vật lý.

Giai đoạn 2: Tìm kiếm:

- Trích xuất đặc trưng: Rút trích vector đặc trưng của ảnh đầu vào cần tìm kiếm.
- Định tuyến: Đo khoảng cách từ vector truy vấn đến K tâm cụm.
- Lựa chọn cụm: Chọn ra tâm cụm có khoảng cách nhỏ nhất. Tra cứu chỉ mục ngược để xuất ra danh sách các Image_ID nằm trong các cụm đó.
- So sánh: Áp dụng thuật toán KNN để tính khoảng cách Cosine chỉ trên nhóm cụm vừa được trích rút.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

- Sắp xếp kết quả: Trả về 5 bức ảnh có độ tương đồng cao nhất làm kết quả cuối cùng.

2.2. K-nearest neighbors (KNN)

KNN (K-Nearest Neighbors) là một trong những thuật toán học có giám sát đơn giản nhất được sử dụng nhiều trong khai phá dữ liệu và học máy.

Khi training, thuật toán này không học một điều gì từ dữ liệu training (lazy learning), mọi tính toán được thực hiện khi nó cần dự đoán kết quả của dữ liệu mới. KNN có thể áp dụng được vào cả hai loại của bài toán Supervised learning là Classification và Regression.

+ Ưu điểm:

- Độ phức tạp tính toán của quá trình training là bằng 0.
- Việc dự đoán kết quả của dữ liệu mới rất đơn giản.
- Không cần giả sử về phân phối của các lớp.

+ Nhược điểm: KNN rất nhạy cảm với nhiễu khi K nhỏ.

- KNN là một thuật toán mà mọi tính toán đều nằm ở khâu test, trong đó việc tính khoảng cách tới từng điểm dữ liệu trong training set sẽ tốn nhiều thời gian. Khi K càng lớn thì độ phức tạp tăng lên.
- Lưu toàn bộ dữ liệu trong bộ nhớ cũng ảnh hưởng tới hiệu năng của KNN.

+ Các bước trong KNN:

- Ta có D là tập các dữ liệu đã gắn nhãn và A là dữ liệu chưa có trong D.
- Đo khoảng cách từ dữ liệu mới A đến tất cả các dữ liệu khác đã được phân loại trong D.
- Chọn K (K là tham số được chọn) khoảng cách nhỏ nhất.
- Kiểm tra danh sách các ảnh và lấy ra K bức ảnh có khoảng cách ngắn nhất trong tập dữ liệu.

Trong thuật toán (KNN) áp dụng cho truy xuất ảnh dựa trên nội dung (Content-Based Image Retrieval - CBIR), tham số K đại diện cho số lượng lân cận gần nhất. Vì vậy, ta chọn K = 5.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

2.2. Truy xuất ảnh dựa trên nội dung

Bản chất cốt lõi của việc truy xuất ảnh là so khớp vector (Vector Matching). Vì mỗi bức ảnh trong CSDL đã được số hóa thành các vector đặc trưng, việc tìm ảnh giống nhau đồng nghĩa với việc tìm các vector có khoảng cách gần nhau nhất trong không gian đa chiều.

Do vector HOG có kích thước là 6084 chiều, trong khi vector HSV chỉ có 192 chiều và vector LBP có 256 chiều nên có chênh lệch lớn, nếu chỉ kết hợp các vector với nhau thì khi tính khoảng cách, sẽ đa số phụ thuộc vào vector HOG mà vector HSV và vector LBP bị lu mờ, làm giảm giá trị của vector đặc trưng.

Vì vậy, ta cần có cơ chế kết hợp vector (Feature Fusion):

Bước 1: Đồng bộ không gian chuẩn hóa

Do vector HOG đã được chuẩn hóa L2, khoảng cách tối đa giữa 2 vector HOG bất kỳ được giới hạn chặt chẽ. Hiện tại, vector HSV và LBP đang được chuẩn hóa L1. Ta cần đưa các vector trên về cùng hệ quy chiếu L2, để khoảng cách sinh ra từ hai vector này sẽ có cùng một thang đo.

Bước 2: Gán trọng số đặc trưng (Weighted Scaling)

Đưa vào các tham số để kiểm soát mức độ đóng góp của hình dạng và màu sắc vào kết quả tìm kiếm cuối cùng. Điều kiện là tổng trọng số phải bằng 1.

Để hệ thống vừa nhận diện được cấu trúc mỏ / đầu / mắt, vừa phân biệt được các phân bố gam màu và kết cấu lông chim một cách tinh tế, ta sẽ thiết lập khởi tạo a (Hình dạng), b (Màu sắc), y (Kết cấu), d (GLCM), e (Hu):

$$V(Fusion) = a.V(HOG) + b.V(HSV) + y.V(LBP) + d.V(GLCM) + e.V(Hu)$$

Bước 3: Nối vector và chuẩn hóa tổng hợp

Ghép nối hai vector đã được hiệu chỉnh trọng số thành một vector đại diện cho bức ảnh: V(Fusion) = [V'(HOG), V'(HSV), V'(LBP), V'(GLCM), V'(Hu)]

Để tối ưu hóa cho thuật toán KNN, việc chuẩn hóa L2 trên vector tổng hợp đảm bảo rằng thuật toán tính khoảng cách sẽ hoạt động trên một mặt cầu đơn vị, giúp kết quả xếp hạng độ tương đồng chính xác hơn.

+ Tính độ tương đồng:

- Độ tương đồng Cosine (Cosine Similarity): Đo góc giữa hai vector, bỏ qua độ lớn

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

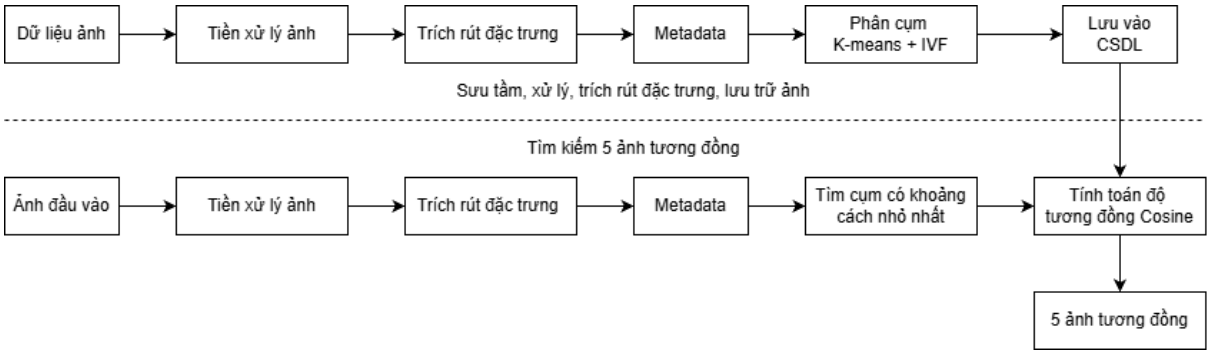
của chúng. Giá trị càng gần 1 thì càng giống nhau:

$$S_C(A,B) = \frac{A \cdot B}{||A|| \times ||B||} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Việc sử dụng độ tương đồng Cosine sẽ giúp hệ thống đánh giá sự tương đồng dựa trên tỷ lệ phân bố của các đặc trưng, loại bỏ sai số do sự chênh lệch kích thước vật thể trong ảnh gây ra.

IV. Xây dựng hệ thống nhận dạng và tìm kiếm ảnh

1. Sơ đồ khối của hệ thống



2. Quy trình thực hiện

Sơ đồ khối bao gồm 12 bước chính, được chia thành 2 giai đoạn:

+ Giai đoạn 1: Sưu tầm, xử lý, trích rút đặc trưng, lưu trữ ảnh.

- Tiền xử lý bộ dữ liệu các ảnh chim (sưu tầm ảnh xóa background, chỉ giữ lại ảnh chim ở giữa khung hình, chụp ngang).

- Chuyển ảnh từ mô hình màu RGB sang mô hình màu HSV / ảnh xám.

- Phân tích và trích rút đặc trưng ảnh.

- Sau khi trích rút đặc trưng là các thuộc tính đã nêu, ta phân cụm các đặc trưng sử dụng K-Means và IVF.

- Lưu trữ thông tin về ảnh (tên file, loài, đường dẫn file) trên CSDL và thông tin đặc trưng của ảnh + thông tin các cụm dưới dạng file vật lý (định dạng .pkl).

+ Giai đoạn 2: Tìm kiếm ảnh tương đồng

- Ảnh truy vấn là ảnh mới, không có trong bộ dữ liệu đã trích xuất đặc trưng.

- Ảnh truy vấn cũng được tiền xử lý (xóa background, cho về giữa khung hình, chuyển từ ảnh đầu vào mô hình màu RGB sang mô hình màu HSV / ảnh xám).

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

- Phân tích trích rút đặc trưng ảnh truy vấn.

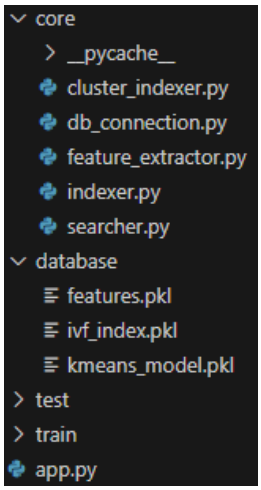
- Tính toán khoảng cách từ ảnh truy vấn đến tâm các cụm, tìm cụm có khoảng cách nhỏ nhất đối với ảnh.

- Tính toán độ tương đồng Cosine với tất cả các ảnh trong cụm đó.

- Chọn ra 5 ảnh có khoảng cách nhỏ nhất và hiển thị kết quả.

3. Cài đặt các bước trung gian

Cấu trúc file của hệ thống:



+ Các thư mục:

- train: Chứa dữ liệu các ảnh được sưu tầm, đưa vào CSDL và huấn luyện.
- test: Chứa dữ liệu các ảnh không có trong tập train, được sử dụng để kiểm thử.
- core: Chứa các file .py, gồm các hàm thực hiện các giai đoạn của sơ đồ khối.
- database: Chứa các file .pkl, gồm các vector đặc trưng, thông tin các cụm được phân bởi K-Means và tên các file trong mỗi cụm.

+ Files:

- db_connection.py: Chứa các hàm thực hiện kết nối và tạo bảng trong SQL Server.
- feature_extractor_py: Chứa các hàm chuyển đổi ảnh, trích xuất đặc trưng và kết hợp các vector lại với nhau thành vector đặc trưng của ảnh.
- indexer.py: Chứa các hàm đẩy dữ liệu về thông tin ảnh (tên file, tên loài, đường dẫn file) lên bảng trong SQL Server, và lưu vector đặc trưng của ảnh lên file features.pkl.
- cluster_indexer.py: Chứa các hàm thực hiện phân cụm theo K-Means và đánh chỉ mục theo IVF (lưu ở 2 file kmeans_model.pkl và ivf_index.pkl).

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

- searcher.py: Chứa các hàm tìm kiếm ảnh tương đồng, được thực hiện như sơ đồ khối.

- app.py: Chứa các hàm tạo giao diện cho hệ thống, thực hiện tìm kiếm ảnh tương đồng dựa vào các hàm đã viết ở các file trên.

3.1. Trích xuất đặc trưng hình ảnh

a. Tiền xử lý ảnh

+ Hàm **read_image(file_path)** sẽ thực hiện đọc ảnh được đưa vào và thực hiện:

- Chuyển đổi ảnh đầu vào sang ảnh có hệ màu RGB.

- Nếu ảnh đầu vào không đúng kích thước 224x224 thì sẽ đưa ảnh về đúng kích thước.

+ Hàm **rgb_to_grayscale(img_rgb):** Chuyển đổi ảnh RGB sang ảnh xám.

+ Hàm **rgb_to_hsv(img_rgb):** Chuyển đổi ảnh RGB sang ảnh có hệ màu HSV.

+ Hàm **create_background_mask(img_gray, threshold=240):** Từ ảnh xám đầu vào, ta tạo ảnh có mặt nạ (Mask) để loại bỏ nền trắng và nhận dạng đối tượng rõ hơn.

```python
# Hàm tiền xử lý ảnh
def read_image(file_path):
    """Đọc ảnh từ đường dẫn và chuyển đổi từ (OpenCV) sang RGB."""
    img_array = np.fromfile(file_path, np.uint8)
    img = cv2.imdecode(img_array, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError(f"Không thể đọc ảnh tại đường dẫn: {file_path}")
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    if img_rgb.shape[:2] != (224, 224):
        img_rgb = cv2.resize(img_rgb, (224, 224))
    return img_rgb

def rgb_to_grayscale(img_rgb):
    """Chuyển đổi ảnh RGB sang ảnh Xám (Grayscale)."""
    return cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)

def rgb_to_hsv(img_rgb):
    """Chuyển đổi ảnh RGB sang không gian màu HSV."""
    return cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)

def create_background_mask(img_gray, threshold=240):
    """
    Tạo mặt nạ (Mask) để loại bỏ nền trắng.
    Pixel nào có cường độ >= 240 (gần trắng) sẽ bị loại bỏ (Mask = 0).
    Pixel thuộc về chim sẽ được giữ lại (Mask = 255).
    """
    _, mask = cv2.threshold(img_gray, threshold, 255, cv2.THRESH_BINARY_INV)
    return mask
```

b. Trích xuất đặc trưng ảnh

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

+ Hàm **extract_hog_features(img_gray):** Tính toán đặc trưng HOG của một ảnh xám. Đầu vào sẽ là một ảnh xám và trả về một vector đặc trưng HOG tương ứng với ảnh đó dưới dạng một mảng.

Quá trình tính toán đặc trưng HOG được thực hiện bằng cách sử dụng hàm hog() của module feature trong thư viện scikit-image và cung cấp các tham số để cấu hình quá trình tính toán, bao gồm: số lượng orientations, kích thước ô pixel, số lượng ô trong mỗi khối, tổng quát hóa căn bậc hai cho các giá trị gradient và chuẩn hóa khối bằng norm L2. Cuối cùng, hàm trả về vector đặc trưng HOG như một mảng.

```
def extract_hog_features(img_gray):
    """Trích xuất vector đặc trưng hình dạng HOG."""
    hog_vector = hog(
        img_gray,
        orientations=9,
        pixels_per_cell=(16, 16),
        cells_per_block=(2, 2),
        block_norm='L2-Hys',
        visualize=False,
        feature_vector=True
    )
    hog_vector = hog_vector / (np.linalg.norm(hog_vector) + 1e-7)
    return hog_vector
```

+ Hàm **extract_hsv_histogram(img_hsv, mask, bins=(16, 4, 3))**: Tính toán histogram của một ảnh đầu vào dựa trên kênh màu HSV, mặt nạ nền và khoảng giá trị. Hàm này nhận vào ba tham số: img_hsv (ảnh đầu vào định dạng HSV), masks (mặt nạ dùng để khử nền) và bins (khoảng giá trị cho từng kênh màu H, S, V).

Quá trình tính histogram được thực hiện bằng cách duyệt qua từng điểm ảnh trong ảnh đầu vào, trích xuất giá trị pixel cho các kênh màu đã chỉ định, sau đó tính toán chỉ mục bin tương ứng cho mỗi pixel dựa trên khoảng giá trị đã cho và kích thước histogram. Cuối cùng, hàm trả về histogram tính được dưới dạng một mảng được chuẩn hóa bằng norm L1.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

```python
def extract_hsv_histogram(img_hsv, mask, bins=(16, 4, 3)):
    """Trích xuất Color Histogram trên hệ màu HSV."""
    hist = cv2.calcHist(
        [img_hsv],
        [0, 1, 2],
        mask,
        [bins[0], bins[1], bins[2]],
        [0, 180, 0, 256, 0, 256]
    )
    hist_vector = hist.flatten()
    cv2.normalize(hist_vector, hist_vector, alpha=1, norm_type=cv2.NORM_L1)
    return hist_vector
```

+ Hàm **extract_lbp_features(img_gray, mask, radius=1):** Tính toán histogram LBP của một ảnh đầu vào dựa trên ảnh xám, mặt nạ nền và khoảng giá trị. Hàm này nhận vào ba tham số: img_gray (ảnh xám), mask (mặt nạ dùng để khử nền) và radius (bán kính dùng để xét các pixel lân cận).

Quá trình tính histogram được thực hiện bằng cách duyệt qua từng điểm ảnh trong ảnh xám, sử dụng hàm **local_binary_pattern(img_gray, n_points, radius, method='default')** của thư viện skimage.feature với tham số lần lượt là ảnh xám đầu vào, số điểm ảnh xung quanh khi quét điểm ảnh qua bán kính, bán kính quét từ điểm ảnh. Tiếp theo, ảnh LBP (mỗi pixel trong ảnh đầu ra chứa giá trị LBP) sẽ được lọc qua mặt nạ, sau đó tính toán histogram được cố định là 256 bins. Cuối cùng, hàm trả về histogram tính được dưới dạng một mảng được chuẩn hóa bằng norm L1.

```python
def extract_lbp_features(img_gray, mask, radius=1):
    """
    Trích xuất đặc trưng kết cấu (Texture) bằng LBP, sử dụng Mask khử nền.
    Đầu ra: Vector kích thước 256 chiều.
    """
    n_points = 8 * radius
    lbp_image = local_binary_pattern(img_gray, n_points, radius, method='default')
    # Lọc qua mặt nạ
    lbp_bird_pixels = lbp_image[mask == 255]
    # Tính Histogram với 256 bins
    hist, _ = np.histogram(lbp_bird_pixels, bins=256, range=(0, 256))
    # Chuẩn hóa L1 cho LBP
    hist_vector = hist.astype("float")
    if hist_vector.sum() > 0:
        hist_vector /= (hist_vector.sum() + 1e-7)
    return hist_vector
```

+ Hàm **extract_glcm_features(img_gray, mask):** Trích xuất đặc trưng kết cấu thống kê bằng ma trận đồng mức xám. Hàm này nhận tham số đầu vào là img_gray (ảnh xám) và mask (mặt nạ dùng để khử nền).

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Đầu tiên, hàm áp dụng trực tiếp mặt nạ lên ảnh xám để đưa về giá trị 0 cho toàn bộ phông nền phía sau con chim. Lúc này ảnh xám chỉ còn hình con chim lơ lửng trên nền đen. Tiếp theo, ta sử dụng hàm **graycomatrix()** với các tham số đầu vào là khoảng cách phân tích các điểm ảnh đứng cạnh nhau (d=1) ở 4 hướng (0, 45, 90, 135 độ). Dữ liệu từ 1 bức ảnh 2 chiều được chuyển hóa thành ma trận 4 chiều kích thước (256, 256, 1, 4).

Vì ta đã đưa nền về màu đen (giá trị 0), ma trận GLCM sẽ ghi nhận có hàng triệu cặp pixel (0, 0) đứng cạnh nhau ở ngoài phông nền. Sự xuất hiện khổng lồ này sẽ gây hiện tượng bias. Hàm sẽ dùng slicing mảng Numpy để cắt bỏ hoàn toàn Hàng 0 và Cột 0 của ma trận để bỏ qua sự ảnh hưởng của phông nền. Mảng này sẽ được chuẩn hóa lại dưới dạng ma trận xác suất.

Sau đó, hàm sẽ sử dụng **graycoprops()** để chiết xuất ra 4 đặc tính vật lý lõi: Contrast (độ tương phản - lông xù xì), Homogeneity (độ đồng nhất - lông mượt mà), Energy (năng lượng vân), và Dissimilarity (độ không tương đồng). Dữ liệu từ 4 mảng 4 chiều (ứng với 4 góc) được ghép lại thành một vector 1 chiều độ dài 16 chiều. Cuối cùng, hàm trả về vector được chuẩn hóa L2 để sẵn sàng nối ghép.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

```python
def extract_glcm_features(img_gray, mask):
    """
    Trích xuất đặc trưng Kết cấu thống kê bằng Ma trận đồng mức xám (GLCM).
    Đầu ra: Vector kích thước 16 chiều.
    """
    # Áp dụng Mask, đưa phông nền trắng về màu đen (giá trị 0)
    masked_gray = cv2.bitwise_and(img_gray, img_gray, mask=mask)
    # Khởi tạo GLCM với khoảng cách d=1 và 4 hướng (0, 45, 90, 135 độ)
    # Normed = False để tự chuẩn hóa sau khi cắt nền
    glcm = graycomatrix(masked_gray, distances=[1], angles=[0, np.pi/4, np.pi/2, 3*np.pi/4],
                        levels=256, symmetric=True, normed=False)
    # Cắt bỏ nhiễu từ phông nền (Loại bỏ hàng 0 và cột 0)
    glcm_bird_only = glcm[1:, 1:, :, :]
    # Chuẩn hóa lại ma trận thành xác suất
    glcm_bird_only = glcm_bird_only.astype('float')
    for d in range(glcm_bird_only.shape[2]):
        for a in range(glcm_bird_only.shape[3]):
            sum_val = glcm_bird_only[:, :, d, a].sum()
            if sum_val > 0:
                glcm_bird_only[:, :, d, a] /= sum_val
    # Trích xuất 4 thuộc tính thống kê (mỗi thuộc tính sinh ra 4 giá trị do có 4 góc)
    contrast = graycoprops(glcm_bird_only, 'contrast').flatten()
    dissimilarity = graycoprops(glcm_bird_only, 'dissimilarity').flatten()
    homogeneity = graycoprops(glcm_bird_only, 'homogeneity').flatten()
    energy = graycoprops(glcm_bird_only, 'energy').flatten()
    # Ghép nối (Concatenate) thành vector 16 chiều và Chuẩn hóa L2
    glcm_vector = np.concatenate([contrast, dissimilarity, homogeneity, energy])
    glcm_vector = glcm_vector / (np.linalg.norm(glcm_vector) + 1e-7)
    return glcm_vector
```

+ Hàm **extract_hu_moments(mask):** Trích xuất đặc trưng hình dạng toàn cục. Hàm này nhận tham số đầu vào là mask (mặt nạ dùng để khử nền).

Quá trình trích xuất đặc trưng được thực hiện bằng cách sử dụng hàm **moments()** để chuyển hóa ảnh mặt nạ thành một cấu trúc từ điển chứa 24 giá trị mômen không gian, đại lượng đo lường sự phân bố khối lượng của các pixel trắng trên hình. Tiếp theo, ta sử dụng hàm **HuMoments()** để tạo ra một mảng 2D kích thước (7, 1), bất biến với phép tịnh tiến, phép xoay và tỷ lệ thu phóng.

Do các giá trị Hu Moments thô thường rất nhỏ (10^(-7), 10^(-12)), hệ thống khó thể đo lường chính xác. Ta chuyển hóa dữ liệu thông qua phép biến đổi Logarit, giúp thu hẹp biên độ dao động của các con số về một khoảng ổn định (VD: từ -7 đến -12), đồng thời hàm giữ nguyên tính chất của hình học gốc. Cuối cùng, hàm trả về vector được chuẩn hóa L2 để nối ghép.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

```python
def extract_hu_moments(mask):
    """
    Trích xuất đặc trưng Hình dạng toàn cục (Global Shape) bằng Hu Moments.
    Đầu ra: Vector kích thước 7 chiều.
    """
    # Tính toán các moment không gian (Spatial Moments)
    moments = cv2.moments(mask)
    # Tính 7 giá trị bất biến Hu Moments
    hu_moments = cv2.HuMoments(moments)
    # Trải phẳng mảng 2D thành vector 1D
    hu_vector = hu_moments.flatten()
    # Chuyển đổi Logarit (Log Scale Transform) để cân bằng biên độ
    for i in range(len(hu_vector)):
        # Áp dụng log10 để giảm kích thước con số, giữ nguyên dấu âm/dương
        if hu_vector[i] != 0:
            hu_vector[i] = -1 * math.copysign(1.0, hu_vector[i]) * math.log10(abs(hu_vector[i]))
    # Chuẩn hóa L2 để đồng bộ thang đo với HOG
    hu_vector = hu_vector / (np.linalg.norm(hu_vector) + 1e-7)
    return hu_vector
```

+ Hàm **feature_fusion():** Kết hợp vector đặc trưng của ảnh dựa trên các vector thuộc tính được trích xuất từ HOG, HSV, LBP, GLCM, Hu. Hàm này nhận vào các tham số: hog_vec (vector được trích xuất từ HOG), hsv_vec (vector được trích xuất từ HSV), lbp_vec (vector được trích xuất từ LBP), glcm_vec (vector được trích xuất từ GLCM), hu_vec (vector được trích xuất từ Hu Moments), các tham số đại diện cho tham số đặc trưng của các vector trên.

Quá trình kết hợp vector đặc trưng được thực hiện bằng cách chuyển đổi vector HSV và LBP về chuẩn hóa norm L2, sau đó gán trọng số cho mỗi vector bằng cách nhân lần lượt mỗi tham số cho mỗi vector thuộc tính được trích xuất ở trên. Cuối cùng, ta nối các vector thuộc tính lại với nhau và hàm trả về vector đặc trưng cuối cùng đã được chuẩn hóa norm L2.

```python
def feature_fusion(hog_vec, hsv_vec, lbp_vec, glcm_vec, hu_vec, alpha=0.2, beta=0.2,
                   gamma=0.2, delta=0.2, epsilon=0.2):
    """
    Kết hợp các vector với trọng số tương ứng.
    """
    # Đưa HSV và LBP về chuẩn L2 để đồng bộ thang đo với HOG
    hsv_vec_l2 = hsv_vec / (np.linalg.norm(hsv_vec) + 1e-7)
    lbp_vec_l2 = lbp_vec / (np.linalg.norm(lbp_vec) + 1e-7)
    # Gán trọng số
    weighted_hog = alpha * hog_vec
    weighted_hsv = beta * hsv_vec_l2
    weighted_lbp = gamma * lbp_vec_l2
    weighted_glcm = delta * glcm_vec
    weighted_hu = epsilon * hu_vec
    # Nối vector (Concatenation)
    combined_vec = np.concatenate([weighted_hog, weighted_hsv, weighted_lbp, weighted_glcm, weighted_hu])
    # Chuẩn hóa L2 lần cuối cho vector tổng hợp
    final_vector = combined_vec / (np.linalg.norm(combined_vec) + 1e-7)
    return final_vector
```

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

+ Hàm **get_image_features():** Hàm tổng hợp thực hiện tiền xử lý ảnh, trích xuất thuộc tính và tạo vector đặc trưng cho ảnh. Hàm này nhận vào các tham số: ảnh đầu vào, các tham số đại diện cho tham số đặc trưng của các vector trên.

Quá trình tạo vector đặc trưng được thực hiện bằng cách đọc và chuyển đổi ảnh sử dụng các hàm **read_image(), rgb_to_grayscale(), rgb_to_hsv()**, tiếp theo là tạo mặt nạ khử nền sử dụng hàm **create_background_mask()**, sau đó hàm thực hiện trích xuất đặc trưng cho các thuộc tính sử dụng hàm **extract_hog_features(), extract_hsv_histogram(), extract_lbp_features(), extract_glcm_features(), extract_hu_moments(),**. Cuối cùng, hàm kết hợp các vector lại với nhau sử dụng hàm **feature_fusion()** và trả về vector đặc trưng cho ảnh đầu vào.

```
def get_image_features(file_path, alpha=0.2, beta=0.2, gamma=0.2, delta=0.2, epsilon=0.2):
    """
    Hàm đóng gói toàn bộ quy trình: Đọc ảnh -> Tiền xử lý -> Trích xuất -> Kết hợp.
    """
    try:
        # Đọc và chuyển đổi ảnh
        img_rgb = read_image(file_path)
        img_gray = rgb_to_grayscale(img_rgb)
        img_hsv = rgb_to_hsv(img_rgb)
        # Tạo mặt nạ khử nền
        mask = create_background_mask(img_gray, threshold=240)
        # Trích xuất đặc trưng
        hog_features = extract_hog_features(img_gray)
        hsv_features = extract_hsv_histogram(img_hsv, mask)
        lbp_features = extract_lbp_features(img_gray, mask)
        glcm_features = extract_glcm_features(img_gray, mask)
        hu_features = extract_hu_moments(mask)
        # Kết hợp vector
        final_feature_vector = feature_fusion(hog_features, hsv_features, lbp_features, glcm_features,
                                              hu_features, alpha, beta, gamma, delta, epsilon)
        return final_feature_vector
    except Exception as e:
        print(f"Lỗi khi trích xuất đặc trưng cho ảnh {file_path}: {e}")
        return None
```

3.2. Phân cụm và đánh chỉ mục ngược

+ Hàm **build_index():** Đây là hàm sử dụng để quét dữ liệu ảnh, thực hiện tạo vector đặc trưng và lưu thông tin ảnh vào CSDL + file vật lý.

Đầu tiên, hàm thiết lập kết nối với SQL Server, sau đó tạo một danh sách rỗng. Tiếp theo, với mỗi ảnh được quét qua trong thư mục, hàm sẽ thực hiện trích xuất thông tin của ảnh (bao gồm tên file, tên loài chim dựa trên tên thư mục của loài đó, đường dẫn tới ảnh) và thực hiện câu truy vấn để lưu thông tin lên CSDL.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Sau đó, hàm thực hiện tạo vector đặc trưng của ảnh sử dụng hàm **get_image_features()** và lưu vector vào danh sách rỗng. Danh sách sẽ được thể hiện dưới dạng từ điển có cấu trúc: {Image_ID: Vector_đặc_trưng}.

Cuối cùng, hàm lưu lại thay đổi vào CSDL và lưu danh sách ra file vật lý (features.pkl) để hệ thống tìm kiếm có thể tái sử dụng mà không phải chạy lại toàn bộ.

```
def build_index():
    """
    Quét thư mục train, trích xuất đặc trưng và lưu vào CSDL & file Pickle.
    """
    # Khởi tạo thư mục và CSDL
    if not os.path.exists(DB_DIR): ···
    init_db()
    conn = get_connection()
    if not conn: ···
    cursor = conn.cursor()
    # Quét thư mục và xử lý ảnh
    features_dict = {} # Dictionary lưu trữ: {Image_ID: Feature_Vector}
    total_processed = 0
    start_time = time.time()
    # Duyệt qua từng thư mục loài chim trong thư mục train
    for species_folder in os.listdir(TRAIN_DIR):
        species_path = os.path.join(TRAIN_DIR, species_folder)
        if not os.path.isdir(species_path):
            continue
        species_label = species_folder # Tên thư mục chính là nhãn loài chim
        # Duyệt qua từng file ảnh trong thư mục
        for img_file in os.listdir(species_path):
            if not img_file.lower().endswith('.png'):
                continue
            file_path = os.path.join(species_path, img_file)
            # Lấy tên file làm Image_ID (Bỏ đuôi .png) - VD: "AFRICAN OYSTER CATCHER (1)"
            image_id = os.path.splitext(img_file)[0]
            # Gọi hàm trích xuất với trọng số đặc trưng
            vector = get_image_features(file_path, alpha=0.5, beta=0.4, gamma=0.1)
            if vector is not None:
                # Lưu vector vào dictionary
                features_dict[image_id] = vector
                # Lưu siêu dữ liệu vào SQL Server
                cursor.execute("""
                    INSERT INTO Bird_Metadata (Image_ID, Species_Label, File_Path)
                    VALUES (?, ?, ?)
                """, (image_id, species_label, file_path))
                total_processed += 1
    # Lưu thay đổi vào CSDL
    conn.commit()
    cursor.close()
    conn.close()
    # Lưu mảng vector ra file Pickle
    with open(FEATURE_FILE, 'wb') as f:
        pickle.dump(features_dict, f)
```

+ Hàm **build_clusters(k=20):** Đây là hàm sử dụng để phân cụm các vector đặc trưng của các ảnh trong file features.pkl, từ đó đánh chỉ mục ngược cho mỗi cụm để thuận tiện trong việc truy vấn. Hàm này nhận tham số đầu vào là k, thể hiện cho số cụm được phân ra dựa trên thuật toán K-Means.

Đầu tiên, hàm đọc file features.pkl dưới chế độ đọc nhị phân và nạp dữ liệu lên bộ nhớ để tính toán. Dữ liệu từ một tệp nhị phân vật lý được chuyển lại thành một đối tượng từ điển có cấu trúc {Image_ID: Vector_đặc_trưng}.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Tiếp theo, hàm sẽ tách ID và vector để tạo 2 mảng riêng biệt đồng bộ về vị trí. Danh sách các vector sẽ được chuyển đổi thành ma trận kích thước 2 chiều N x D, với N là số lượng ảnh và D là số chiều của vector.

```
def build_clusters(k=20):
    """
    Đọc vector từ CSDL, chạy thuật toán K-Means gom cụm và tạo Chỉ mục ngược (IVF).
    """
    # Đọc dữ liệu vector đã trích xuất
    if not os.path.exists(FEATURE_FILE):
        print(f"Không tìm thấy file {FEATURE_FILE}. Hãy chạy indexer.py trước.")
        return
    with open(FEATURE_FILE, 'rb') as f:
        features_dict = pickle.load(f)
    if not features_dict:
        print("Dữ liệu vector trống.")
        return
    # Chuẩn bị ma trận dữ liệu cho thuật toán K-Means
    list_ids = list(features_dict.keys())
    list_vectors = list(features_dict.values())
    # Chuyển đổi thành ma trận Numpy (N x D), trong đó N là số ảnh, D là số chiều vector
    X = np.array(list_vectors)
    print(f"Kích thước ma trận dữ liệu: {X.shape[0]} ảnh x {X.shape[1]} chiều.")
```

Sau đó, hàm sử dụng thuật toán K-Means để gom N điểm dữ liệu trong không gian D chiều thành k cụm dựa trên độ tương đồng (khoảng cách) của chúng. Khởi tạo **KMeans(n_clusters=k, random_state=42, n_init=10)** với tham số đầu vào là số cụm được phân ra, random_state và n_init giúp kết quả phân cụm được cố định qua mỗi lần chạy. Khi huấn luyện thành công, đầu ra thu được mô hình K-Means mang theo hai luồng thông tin: Ma trận tọa độ của k tâm cụm và một mảng nhãn phân loại có chiều dài N, lưu trữ số thứ tự cụm mà mỗi bức ảnh được gắn vào.

Tiếp theo, hàm tạo một chỉ mục ánh xạ trực tiếp từ định danh cụm (cluster_id) ra danh sách các bức ảnh thuộc về cụm đó để truy xuất. Khởi tạo dictionary rỗng cho k cụm, duyệt qua mảng nhãn và ghép nối ID ảnh vào cụm tương ứng.

Cuối cùng, hàm trả về kết quả của quá trình phân cụm và lập chỉ mục ra file vật lý để hệ thống tìm kiếm có thể tái sử dụng mà không phải chạy lại thuật toán.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

```
# Sử dụng n_init='auto' và random_state=42 để kết quả phân cụm luôn cố định qua các lần chạy
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
kmeans.fit(X)
# Xây dựng Chỉ mục ngược (Inverted Index)
ivf_index = {}
for i in range(k):
    ivf_index[i] = [] # Khởi tạo danh sách rỗng cho từng cụm
# Duyệt qua các nhãn cụm (kmeans.labels_) và đưa ID ảnh vào đúng cụm của nó
labels = kmeans.labels_
for i, cluster_id in enumerate(labels):
    image_id = list_ids[i]
    ivf_index[cluster_id].append(image_id)
# In thống kê phân bố ảnh trong các cụm
print("Thống kê phân bố ảnh trong các cụm:")
for cluster_id, images_in_cluster in ivf_index.items():
    print(f"   - Cụm {cluster_id:<2}: {len(images_in_cluster):<3} ảnh")
# Lưu Model và IVF
with open(KMEANS_MODEL_FILE, 'wb') as f:
    pickle.dump(kmeans, f)
with open(IVF_INDEX_FILE, 'wb') as f:
    pickle.dump(ivf_index, f)
```

3.3. Tìm kiếm ảnh tương đồng

+ Hàm **__init__(self):** Khởi tạo bộ tìm kiếm bằng cách tải toàn bộ cơ sở dữ liệu và mô hình từ file vật lý lên bộ nhớ. Việc này giúp hệ thống truy xuất dữ liệu theo thời gian thực mà không bị độ trễ do thao tác đọc file lặp lại.

Hàm sẽ lần lượt đọc các file vật lý dưới dạng đọc nhị phân, sau đó gắn cho mỗi đối tượng trên bộ nhớ với một file. Danh sách vector đặc trưng sẽ có cấu trúc {Image_ID: Vector_đặc_trưng}; Mô hình K-Means chứa chứa thông tin về tọa độ của K tâm cụm; Danh sách chỉ mục ngược sẽ có cấu trúc {Cluster_ID: [Danh_sách_Image_ID]}.

```
def __init__(self):
    """Khởi tạo bộ tìm kiếm: Tải Vector, Mô hình K-Means và Chỉ mục IVF vào RAM."""
    try:
        # Tải bộ vector đặc trưng (Dùng để tra cứu sau khi đã xác định được cụm)
        with open(FEATURE_FILE, 'rb') as f:
            self.features_db = pickle.load(f)
        # Tải Mô hình K-Means (Chứa thông tin các Tâm cụm)
        with open(KMEANS_MODEL_FILE, 'rb') as f:
            self.kmeans_model = pickle.load(f)
        # Tải Chỉ mục ngược IVF (Inverted File Index)
        with open(IVF_INDEX_FILE, 'rb') as f:
            self.ivf_index = pickle.load(f)
    except FileNotFoundError as e:
        print(f"Lỗi: Không tìm thấy file dữ liệu. Chi tiết: {e}")
        self.features_db = None
        self.kmeans_model = None
        self.ivf_index = None
```

+ Hàm **search(self, query_vector, k=5, n_probe=1):** Nhận vector đặc trưng của bức ảnh truy vấn, sau đó sử dụng kỹ thuật K-Means + IVF để khoanh vùng và đối chiếu, từ đó trả về 5 ảnh tương đồng nhất. Hàm này nhận tham số đầu vào là vector đặc trưng

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

của ảnh truy vấn, k là số ảnh tương đồng được trả về, n_probe là số cụm lân cận được mở ra để tìm kiếm.

Đầu tiên, hàm sẽ chuyển hóa vector ảnh truy vấn thành ma trận 2 chiều kích thước 1xD nhằm tương thích với chuẩn đầu vào của thư viện scikit-learn. Hàm tính khoảng cách từ vector này đến toàn bộ các tâm cụm, đưa ra là một mảng 1D chứa các giá trị khoảng cách, rồi sắp xếp mảng tăng dần và lấy ra chỉ số của n_probe cụm có khoảng cách nhỏ nhất.

Tiếp theo, hàm sẽ khởi tạo danh sách rỗng chứa ID của các ảnh trong cụm. Hàm duyệt qua các cụm gần nhất, tra cứu cluster_id trong danh sách chỉ mục ngược để rút danh sách ID ảnh và nối vào danh sách rỗng. Ta ép kiểu qua tập hợp (set) nhằm loại bỏ ID ảnh trùng lặp, sau đó ép ngược lại thành danh sách (list). Danh sách lúc này sẽ chứa Image_ID của các ảnh tiềm năng nhất.

```
def search(self, query_vector, k=5, n_probe=1):
    """
    Tìm kiếm Top K ảnh tương đồng sử dụng K-Means và IVF.
    :param query_vector: Vector đặc trưng của ảnh đầu vào (1D Array).
    :param k: Số lượng ảnh trả về.
    :param n_probe: Số lượng cụm lân cận sẽ được mở ra để tìm kiếm.
    """
    if not self.kmeans_model or not self.ivf_index or not self.features_db:
        print("Hệ thống chưa được khởi tạo đúng cách.")
        return []
    # XÁC ĐỊNH CỤM GẦN NHẤT (ROUTING)
    # Đưa vector 1D thành ma trận 2D (1 x D) để tương thích với thư viện scikit-learn
    query_vec_2d = query_vector.reshape(1, -1)
    # Hàm transform() tính khoảng cách từ vector truy vấn đến TẤT CẢ các tâm cụm
    distances_to_clusters = self.kmeans_model.transform(query_vec_2d)[0]
    # Lấy ra ID của `n_probe` cụm có khoảng cách nhỏ nhất (Ví dụ: 2 cụm gần nhất)
    nearest_clusters = np.argsort(distances_to_clusters)[:n_probe]

    # THU HẸP KHÔNG GIAN TÌM KIẾM (CANDIDATE SELECTION)
    candidate_ids = []
    for cluster_id in nearest_clusters:
        # Tra cứu từ điển IVF để lấy ID các ảnh nằm trong cụm đó
        candidate_ids.extend(self.ivf_index[cluster_id])
    # Loại bỏ các ID trùng lặp (nếu có)
    candidate_ids = list(set(candidate_ids))
    # In log ra terminal để quan sát sự tối ưu hóa không gian tìm kiếm
    print(f"[IVF Search] Mở {n_probe} cụm lân cận gần nhất: {nearest_clusters}")
```

Sau đó, hàm khởi tạo danh sách rỗng chứa độ tương đồng từ ảnh truy vấn đến các ảnh trong danh sách chứa Image_ID ở trên. Sau khi đo chiều dài vector ảnh truy vấn, hàm sẽ duyệt qua từng Image_ID trong danh sách ứng viên, trích xuất vector đặc trưng tương ứng khi đọc file features.pkl và tính độ tương đồng Cosine. Ta chuyển

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

hóa độ tương đồng thành khoảng cách bằng 1.0 - độ tương đồng, và lưu kết quả dưới dạng tuple (img_id, distance) vào danh sách rỗng.

Cuối cùng, hàm sắp xếp danh sách tăng dần dựa trên giá trị khoảng cách, lấy 5 kết quả đầu tiên và gọi hàm **_fetch_metadata_from_sql** để lấy thông tin chi tiết của 5 bức ảnh trên.

```
# TÍNH KHOẢNG CÁCH TRONG CỤM
results = []
# Vì vector đã chuẩn hóa L2, nên np.linalg.norm xấp xỉ 1.
# Chuẩn hóa lại ở mẫu số để đảm bảo triệt tiêu hoàn toàn sai số dấu phẩy động
query_norm = np.linalg.norm(query_vector)
for img_id in candidate_ids:
    if img_id not in self.features_db: continue
    db_vector = self.features_db[img_id]
    # Tính độ tương đồng Cosine
    db_norm = np.linalg.norm(db_vector)
    cosine_similarity = np.dot(query_vector, db_vector) / ((query_norm * db_norm) + 1e-7)
    # Chuyển thành khoảng cách
    distance = 1.0 - cosine_similarity
    distance = max(0.0, distance)
    results.append((img_id, distance))
# Sắp xếp tăng dần theo khoảng cách (Khoảng cách càng nhỏ càng giống)
results.sort(key=lambda x: x[1])
top_k_results = results[:k]
# LẤY METADATA TỪ SQL SERVER
return self._fetch_metadata_from_sql(top_k_results)
```

+ Hàm **_fetch_metadata_from_sql(self, top_k_results):** Giao tiếp với SQL Server để lấy các thông tin dạng văn bản (nhãn loài chim, đường dẫn file ảnh trên ổ cứng) dựa trên danh sách k Image_ID vừa tìm được. Đầu vào sẽ là danh sách tuple chứa cấu trúc (Image_ID, distance) vừa thực hiện ở hàm **search()**.

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

Đầu tiên, hàm thiết lập kết nối với SQL Server, sau đó tách lấy riêng danh sách các Image_ID. Tiếp theo, hàm thực hiện câu lệnh truy vấn và thu về kết quả dưới dạng danh sách các dòng dữ liệu. Hàm sẽ chuyển danh sách này thành từ điển trung gian với cấu trúc {Image_ID: {'label': nhãn, 'path': đường dẫn}} giúp việc tra cứu thông tin diễn ra tức thời.

Cuối cùng, hàm khởi tạo danh sách rỗng, duyệt qua danh sách đầu vào. Với mỗi Image_ID, hàm lấy label và path, làm tròn khoảng cách đến 4 chữ số thập phân, rồi đóng gói tất cả thành một từ điển hoàn chỉnh cho mỗi kết quả. Hàm sẽ trả về một mảng chứa các từ điển gồm đủ 4 trường thông tin: Tên file, khoảng cách, loài và đường dẫn tới ảnh đó.

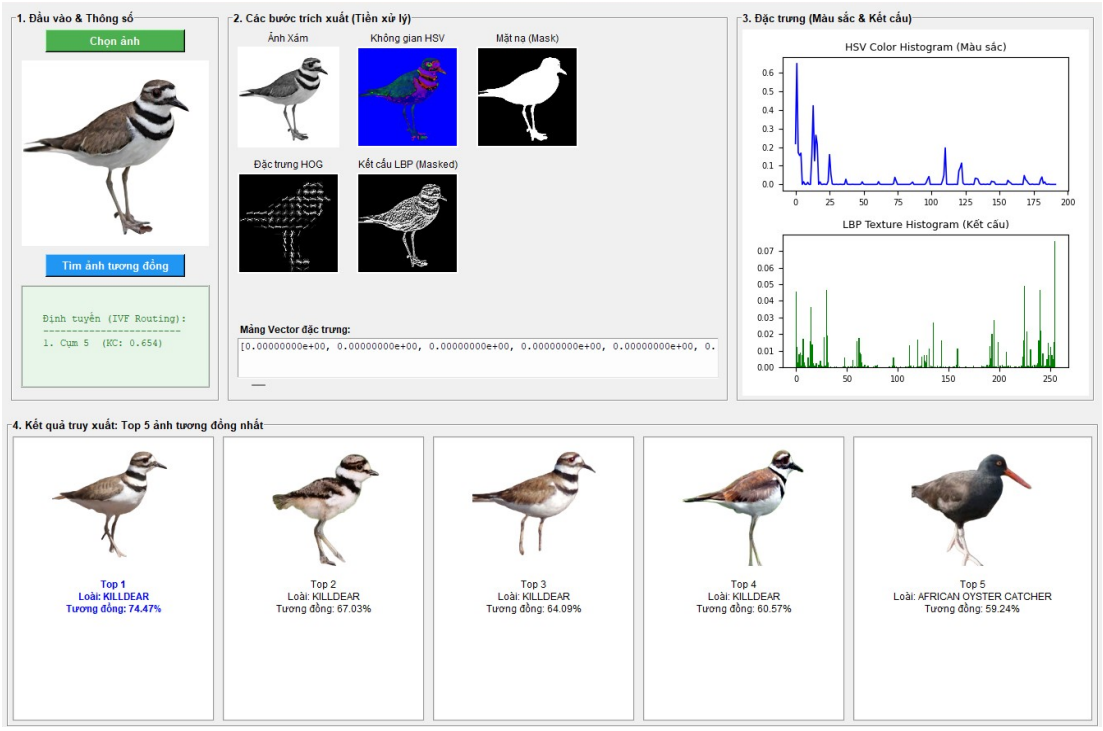
```
def _fetch_metadata_from_sql(self, top_k_results):
    """Truy vấn SQL Server để lấy đường dẫn và tên loài chim cho Top K ID."""
    conn = get_connection()
    if not conn:
        print("Không thể kết nối SQL Server để lấy Metadata.")
        return []
    top_k_ids = [item[0] for item in top_k_results]
    if not top_k_ids:
        return []
    placeholders = ','.join(['?'] * len(top_k_ids))
    query = f"SELECT Image_ID, Species_Label, File_Path FROM Bird_Metadata WHERE Image_ID IN ({placeholders})"
    cursor = conn.cursor()
    cursor.execute(query, top_k_ids)
    rows = cursor.fetchall()
    # Gắn metadata vào dictionary
    db_data = {row[0]: {'label': row[1], 'path': row[2]} for row in rows}
    # Tạo danh sách kết quả cuối cùng (Giữ đúng thứ tự đã xếp hạng của thuật toán)
    final_results = []
    for image_id, dist in top_k_results:
        if image_id in db_data:
            final_results.append({
                'image_id': image_id,
                'distance': round(dist, 4),
                'species': db_data[image_id]['label'],
                'file_path': db_data[image_id]['path']
            })
    cursor.close()
    conn.close()
    return final_results
```

V. Kết quả kiểm nghiệm

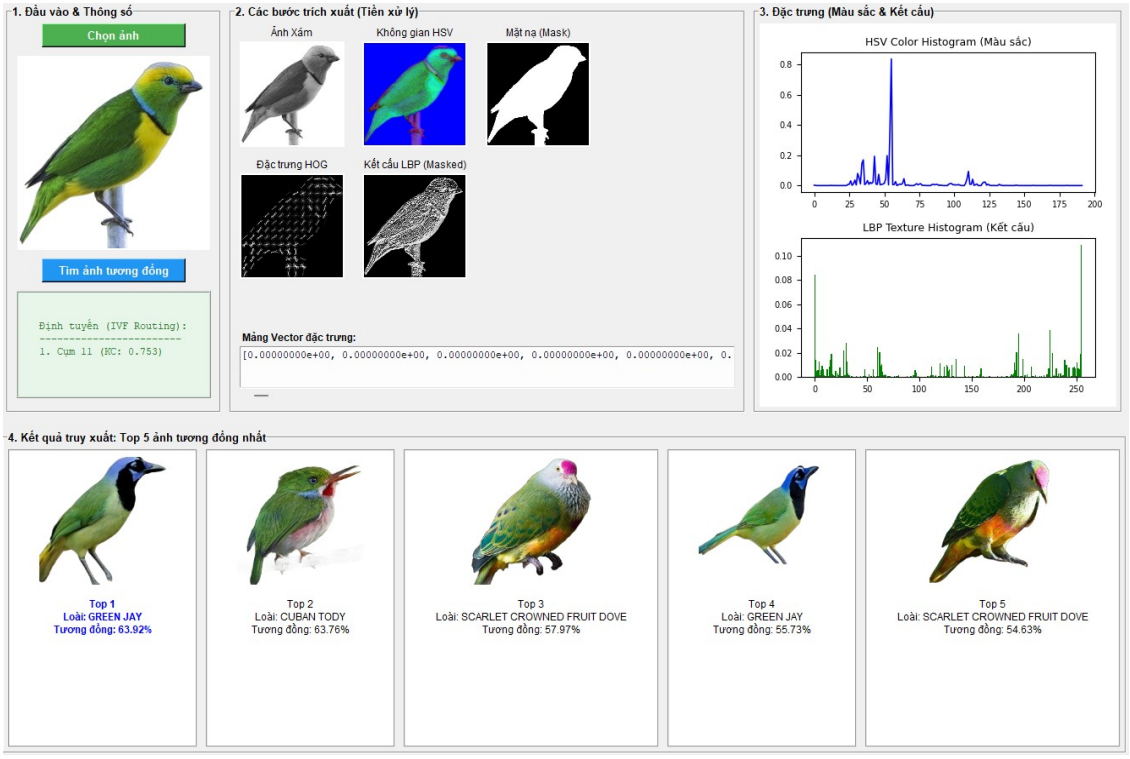
1. Kết quả quá trình tìm kiếm

- Với ảnh đầu vào là ảnh của loài có trong dữ liệu (VD: Loài KILLDEAR):

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)



- Với ảnh đầu vào là ảnh của loài không có trong dữ liệu (VD: Loài GOLDEN CHLOROPHONIA):



2. Đánh giá kết quả

Việc đánh giá kết quả dựa trên số lượng ảnh đoán đúng với một số loài chim trong tập ảnh đầu vào. Ta sẽ chính chỉ số Precision = Số ảnh đoán cùng loài / (Số ảnh

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)

đoán cùng loài + Số ảnh đoán khác loài). Kết quả tổng hợp sẽ bằng bằng độ chính xác trung bình và thời gian truy vấn trung bình cho mỗi ảnh.

Tên loài chim	Số ảnh Test	Precision@5
AFRICAN OYSTER CATCHER	6	73.33%
BALTIMORE ORIOLE	5	88.00%
CASPIAN TERN	7	91.43%
CUBAN TODY	6	73.33%
GREEN JAY	6	73.33%
GREEN WINGED DOVE	5	52.00%
KILLDEAR	5	84.00%
LAZULI BUNTING	5	44.00%
MALACHITE KINGFISHER	10	100.00%
PAINTED BUNTING	5	100.00%
PARADISE TANAGER	5	52.00%
PINK ROBIN	5	64.00%
RUFUOS MOTMOT	6	43.33%
SCARLET CROWNED FRUIT DOVE	5	48.00%
TREE SWALLOW	6	63.33%
TURKEY VULTURE	6	73.33%
WOODLAND KINGFISHER	6	80.00%

Tổng số ảnh truy vấn (Test) : 99 ảnh
Độ chính xác trung bình (mAP): 72.53%
Thời gian truy vấn trung bình: 10.42 ms / ảnh

studeersnel
Scan to open on Studeersnel
Downloaded by Tr?n M?nh Khiêm (khiemmanhtran@gmail.com)